

$\theta > A$

搜索树应用

区间树

Your instinct, rather than precision stabbing, is more about just random bludgeoning.

没有什么能够腐蚀那条钢带...音乐刺穿这些模糊的形体, 从它们中间越过

邓俊辉

deng@tsinghua.edu.cn

穿刺查询/ stabbing Query

❖ 在x轴上给定一组区间:

$$S = \{ s_i = [x_i, x'_i] \mid 1 \leq i \leq n \}$$

对于任何点 q_x , 筛选出所有包含它的区间:

$$S(q_x) = \{ s_i = [x_i, x'_i] \mid x_i \leq q_x \leq x'_i \}$$

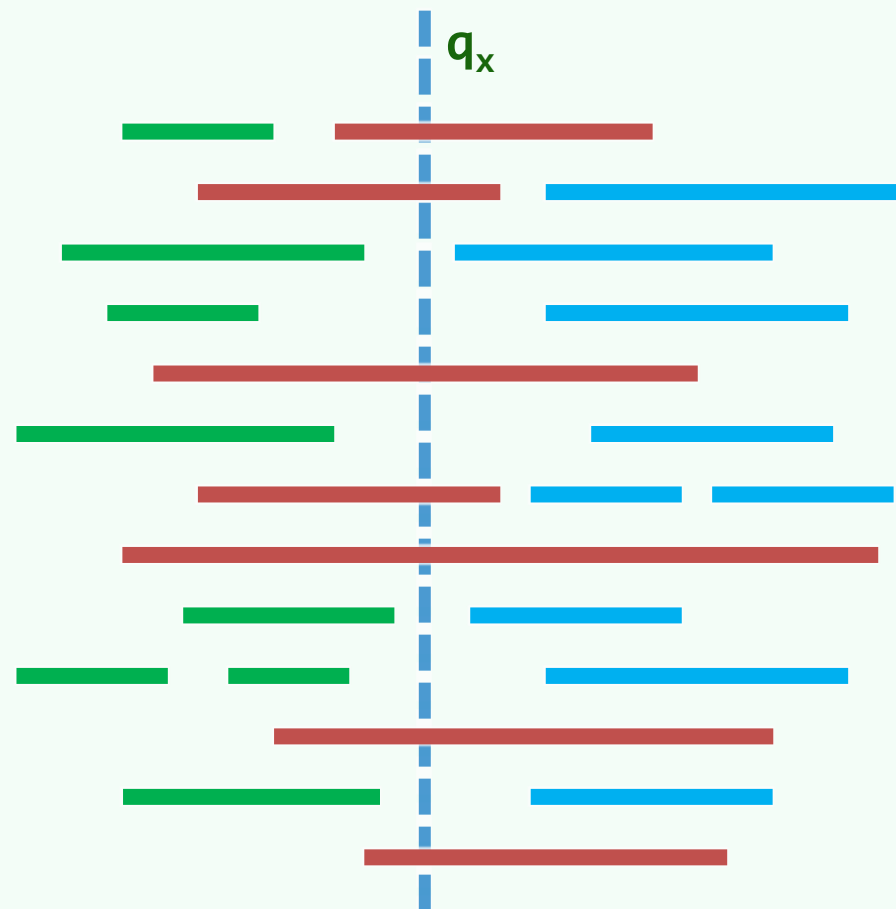
❖ 蛮力解法不值一提

未能利用区间集相对**固定**的条件

❖ 而将区间集预处理成一棵区间树| Interval Tree

可以得到一个**输出敏感的在线**查询算法

❖ 为理解其原理, 不妨先从**简单情况**入手...



假如，所有区间的交非空

❖ 取它们的一个公共点 $m \in \bigcap_{i=1}^n s_i \neq \emptyset$

❖ 作为预处理，可按由远及近的次序
将所有的左端点、右端点分别排序

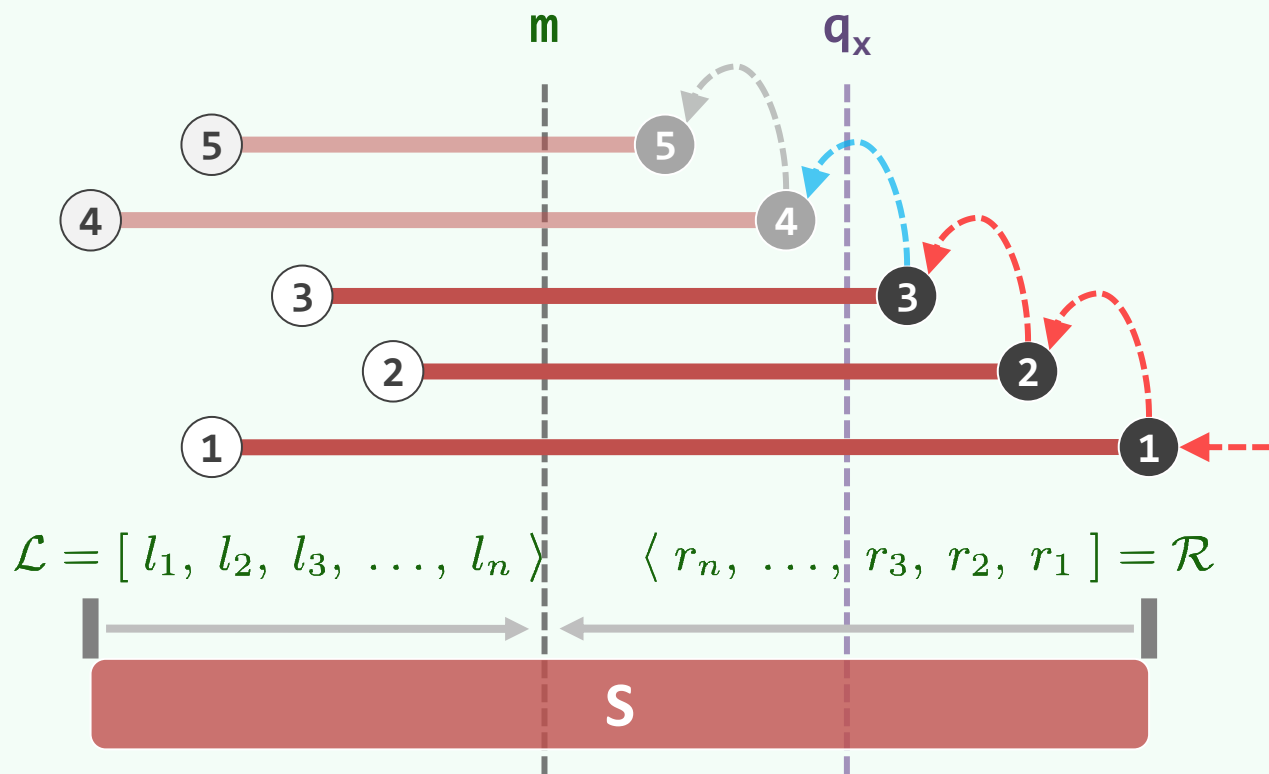
❖ 此后，对任意穿刺位置 q_x 都可快捷处理

WLOG, 假定 $m < q_x$

❖ 观察：所有区间的左端点都在 m 及 q_x 以左

因此，区间是否被刺中，仅取决于其右端点是否足够靠右（远）

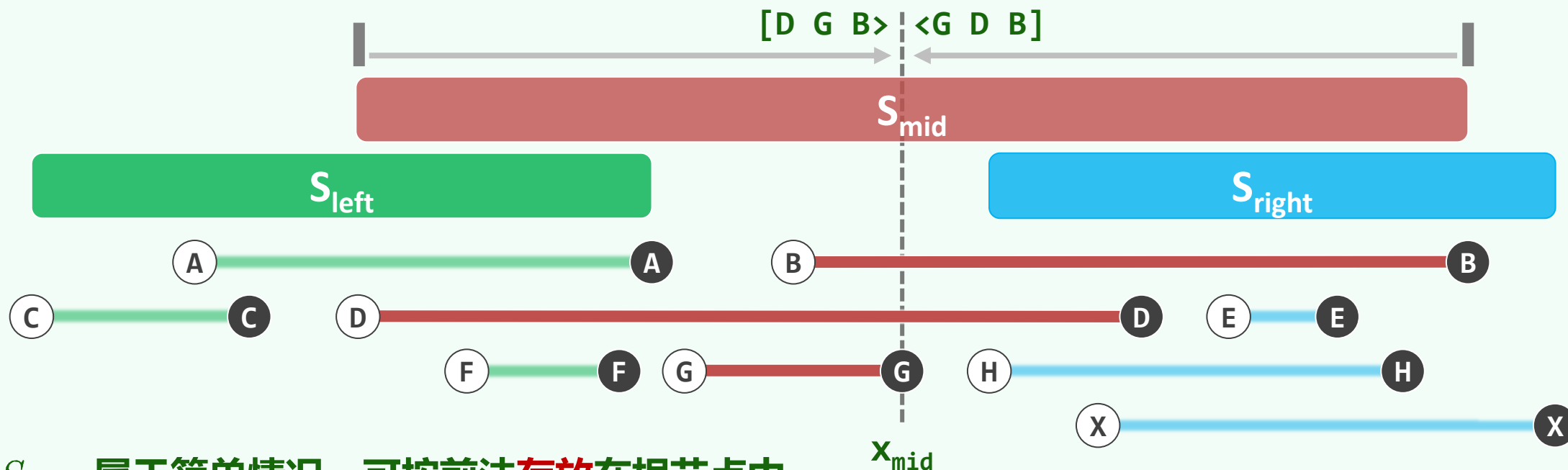
❖ 于是，只需取出右端点的排序序列，由远及近地逐个检视： $\mathcal{O}(r + 1)$



分而治之

❖ 以所有端点的中位点 x_{mid} 为界，将所有区间划分为三类：

$$S_{left} = \{ S_i \mid x'_i < x_{mid} \} \quad S_{mid} = \{ S_i \mid x_i \leq x_{mid} \leq x'_i \} \quad S_{right} = \{ S_i \mid x_{mid} < x_i \}$$



❖ S_{mid} 属于简单情况，可按前法存放在根节点中

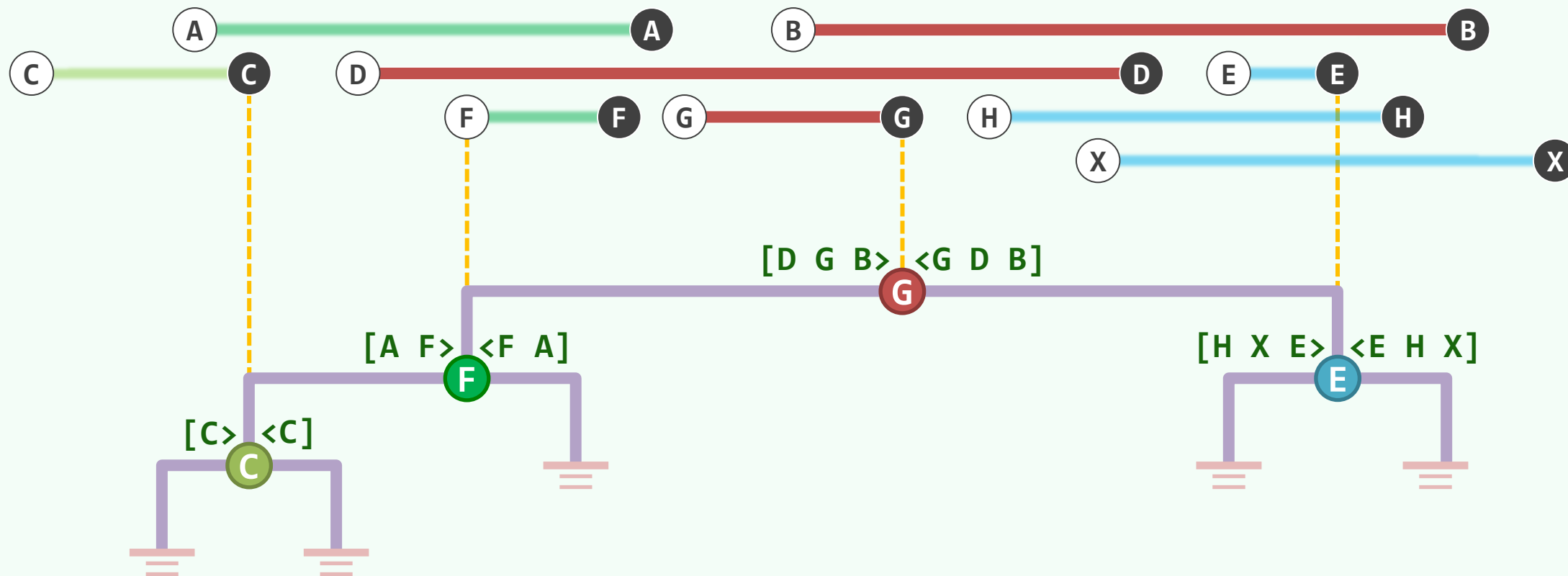
❖ S_{left} 和 S_{right} 则交给左、右子树：将递归地同样划分，直至无需划分

渐近平衡: $O(\log n)$ 深度

$$\max\{|S_{left}|, |S_{right}|\} \leq n/2$$

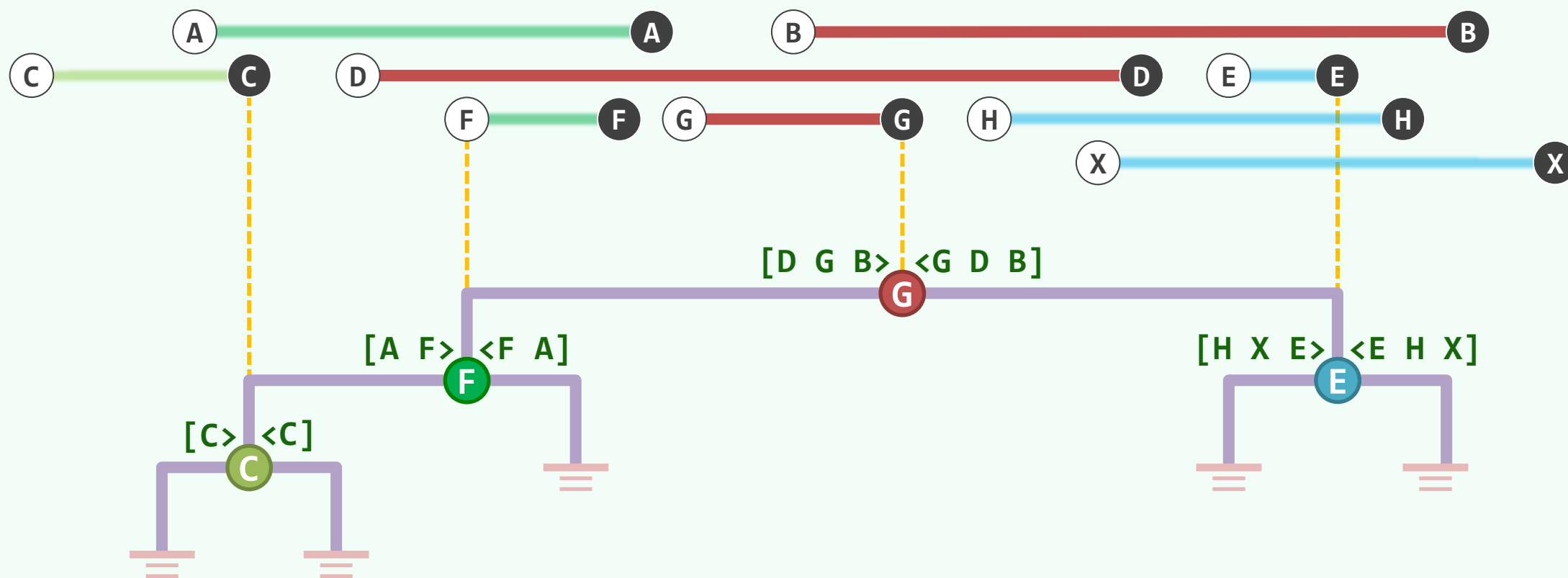
Best case: $|S_{mid}| = n$

Worst case: $|S_{mid}| = 1$



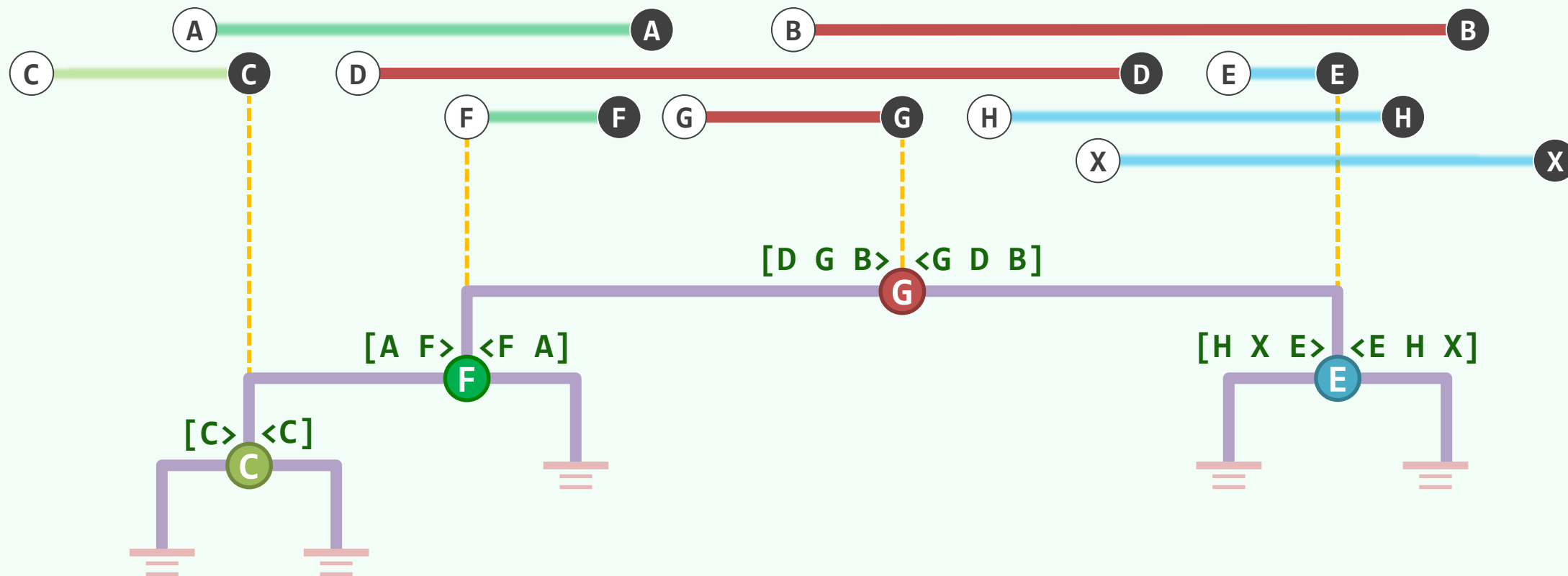
关联列表 | Associative Lists

❖ S_{mid} 中的区间, 分别按左、右端点的次序, 由外向内地排成一对列表: $\mathcal{L} \mid \mathcal{R}$



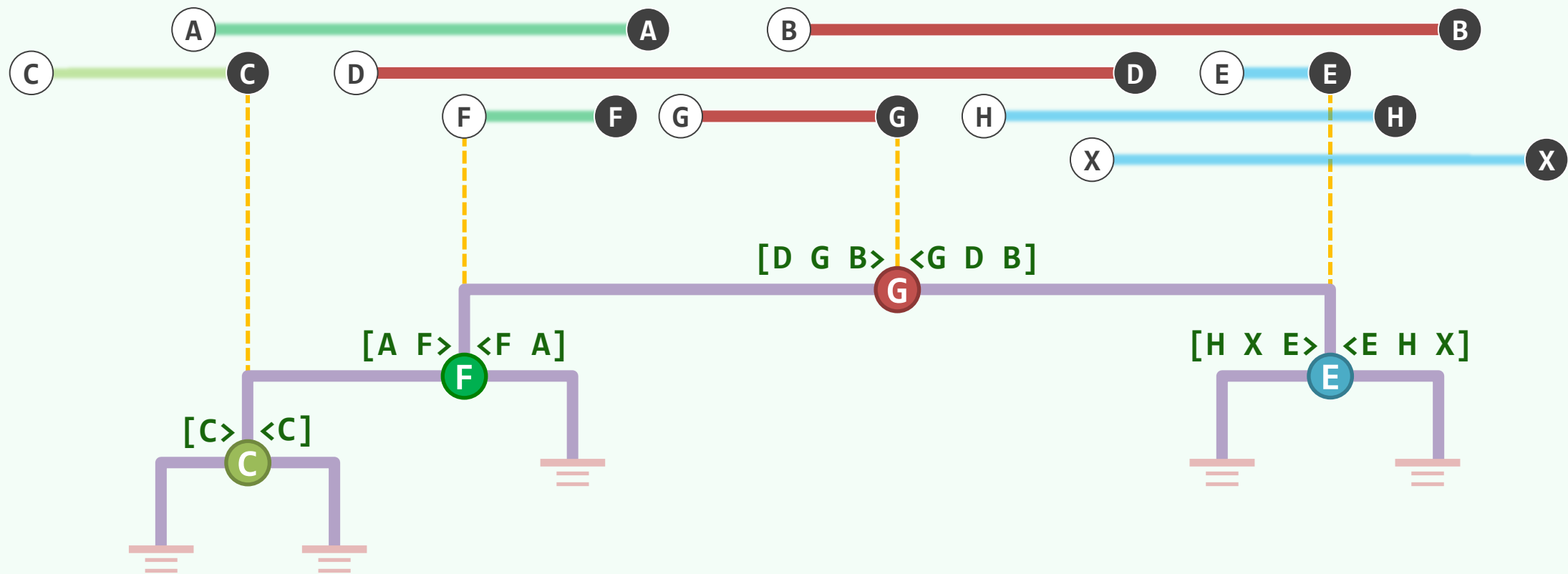
总共占用 $O(n)$ 空间

❖ 该树自身占用 $O(n)$ 空间；输入在每个区间各存放了**两份**，总共也是 $O(n)$



构造只需 $O(n \log n)$ 时间

❖ 为此，需要避免反复地对端点排序...



查询算法: $\text{queryIntTree}(v, q_x)$

if (! v) return //递归基

if ($q_x < x_{\text{mid}}(v)$)

借助 $\mathcal{L}$, 从 $S_{\text{mid}}(v)$ 中分拣出包含 q_x 的区间

$\text{queryIntTree}(\text{lc}(v), q_x)$ //rc(v)被剪枝

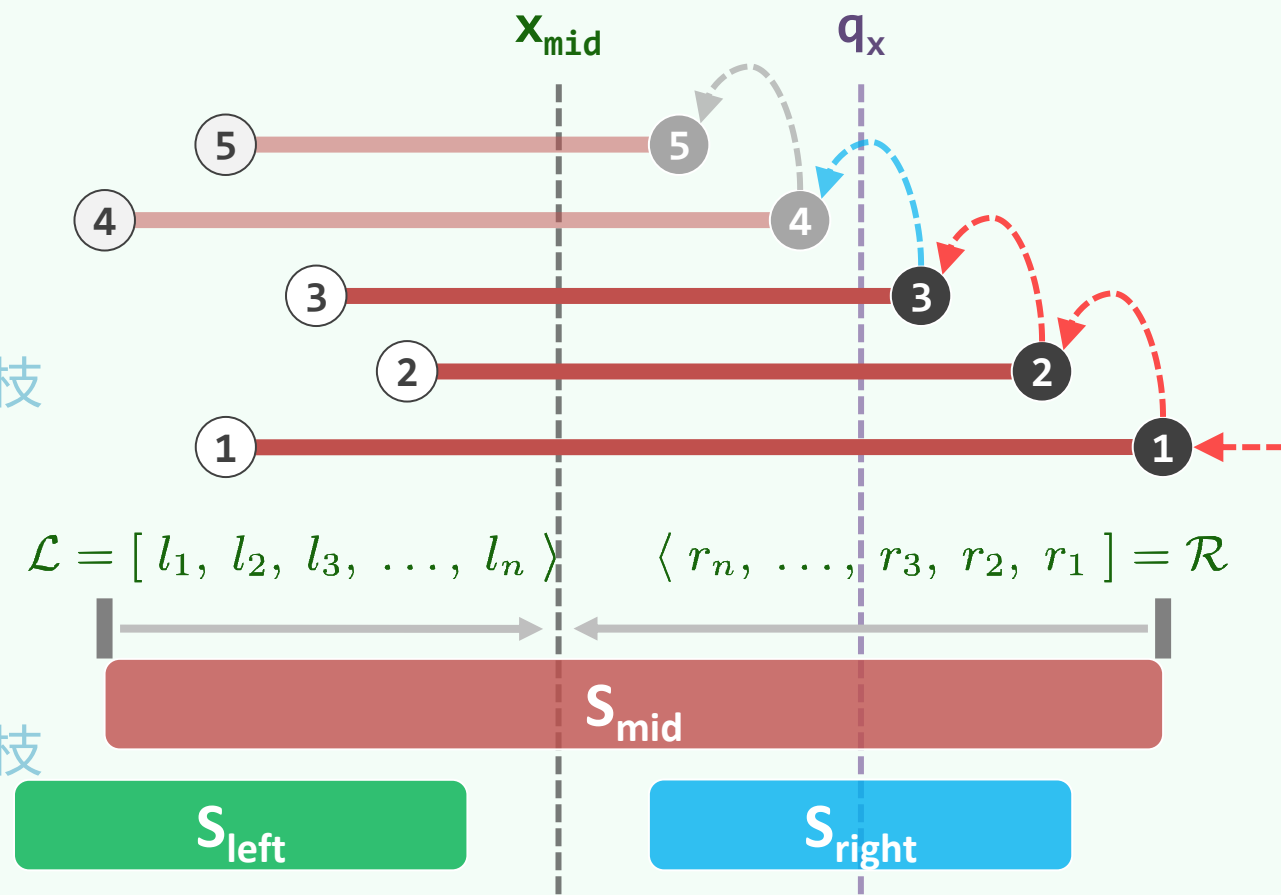
else if ($x_{\text{mid}}(v) < q_x$)

借助 $\mathcal{R}$, 从 $S_{\text{mid}}(v)$ 中分拣出包含 q_x 的区间

$\text{queryIntTree}(\text{rc}(v), q_x)$ //lc(v)被剪枝

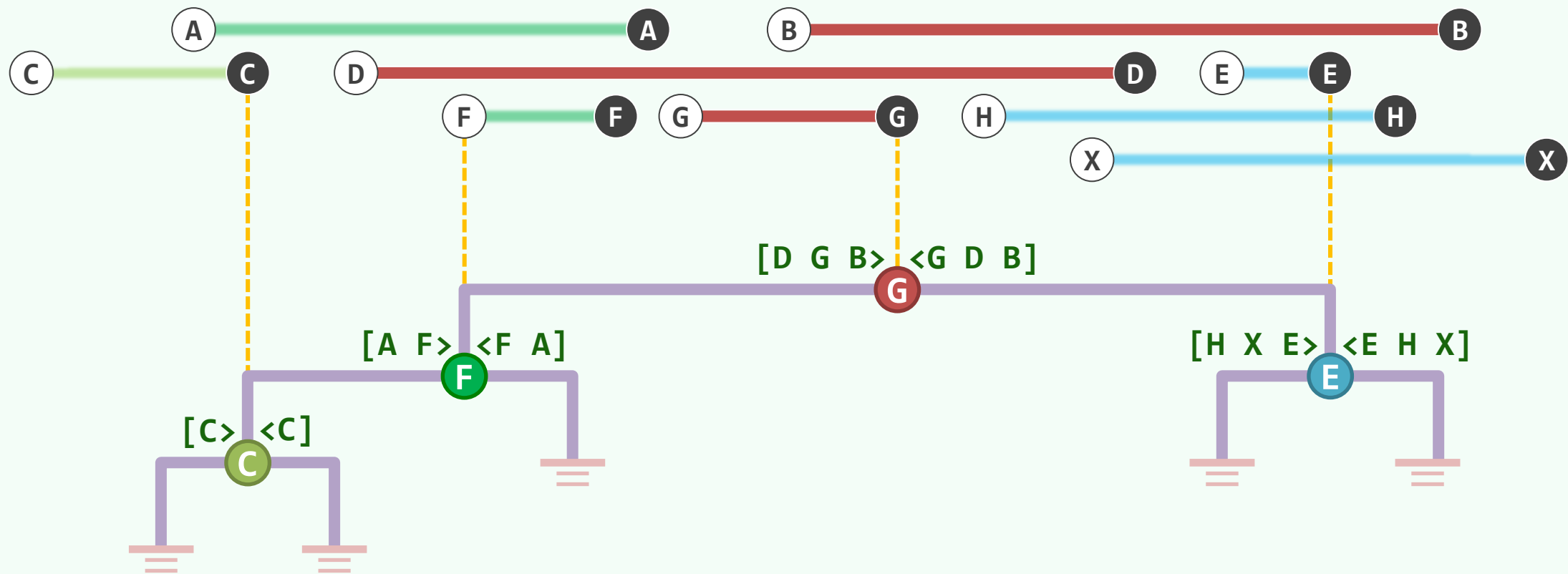
else //概率无限接近于0

直接报告 $S_{\text{mid}}(v)$



$O(r + \log n)$ 查询时间

❖ 线性递归，只会访问 $O(\log n)$ 个节点；对 L/R-List 的访问：累计 $O(r)$ 个元素成功， $O(\log n)$ 个失败





搜索树应用

线段树

邓俊辉

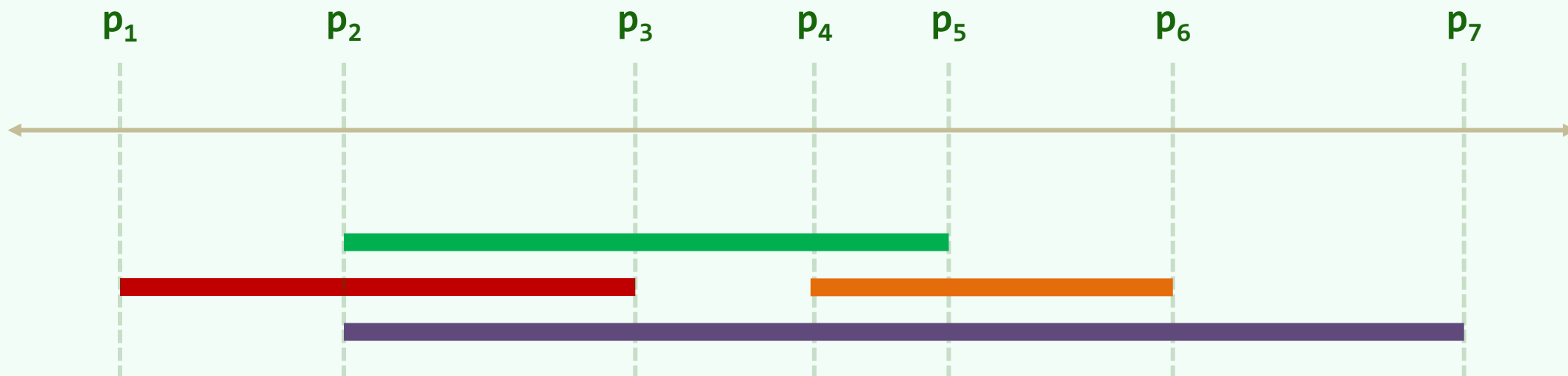
deng@tsinghua.edu.cn

把一条线分割成不相等的两段，再把这两段按照同样的比例再分成两个部分。假设第一次分出来的两段中，一段代表可见世界，另一段代表理智世界。然后再看第二次分成的两段，他们分别代表清楚与不清楚的程度，你便会发现，可见世界那一段的第一部分是它的影像

初级区间/Elementary Interval

❖ 任给x-轴上的n段区间: $I = \{ [x_i, x'_i] \mid i = 1, 2, 3, \dots, n \}$

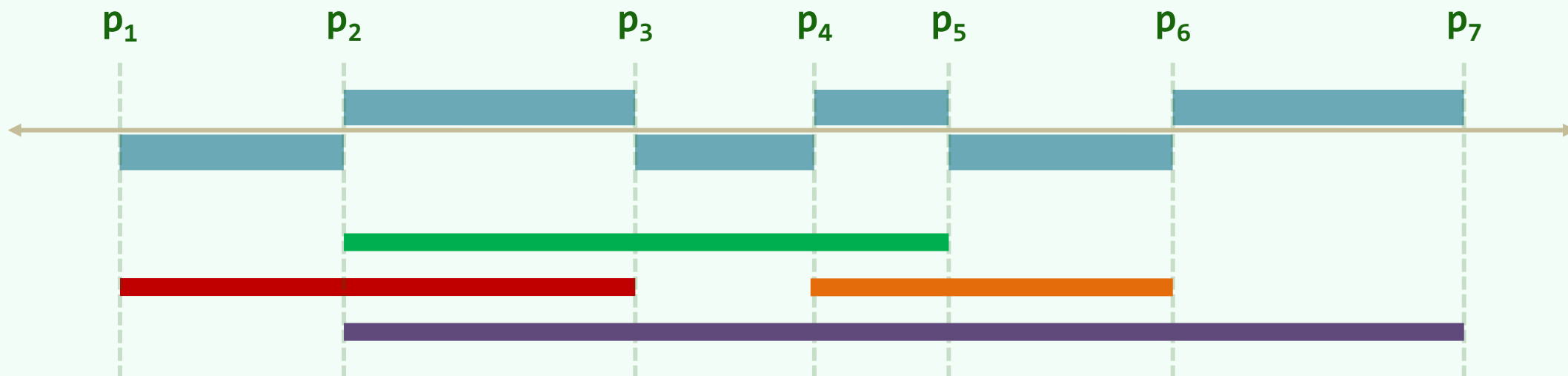
❖ 将它们的端点排序为: $\{ p_1, p_2, p_3, \dots, p_m \}, m \leq 2n$



❖ 于是, 便得到m-1段初级区间/EI: $(p_1, p_2], (p_2, p_3], \dots, (p_{m-1}, p_m]$

离散化/Discretization

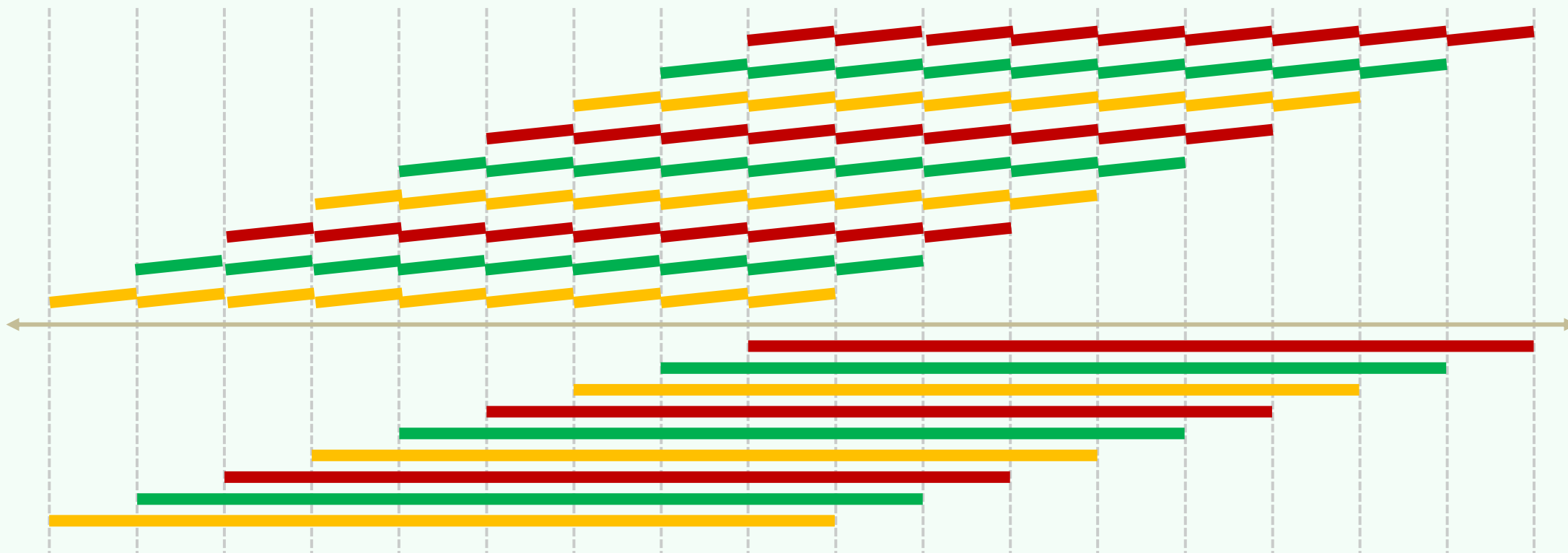
- ❖ 观察：同一段EI内的任何位置，穿刺查询的输出必然完全一样
- ❖ 于是，只要将所有EI预处理为有序向量，并令各段EI预先记录其对应的查询输出，那么...



- ❖ ...一旦确定穿刺位置，便可快速地完成查询：二分查找 + 输出结果 $O(\log n + r)$

空间爆炸

- ❖ 在最坏情况下，输入的每段区间都会横跨 $\Omega(n)$ 段EI，总共需要 $\Omega(n^2)$ 附加空间



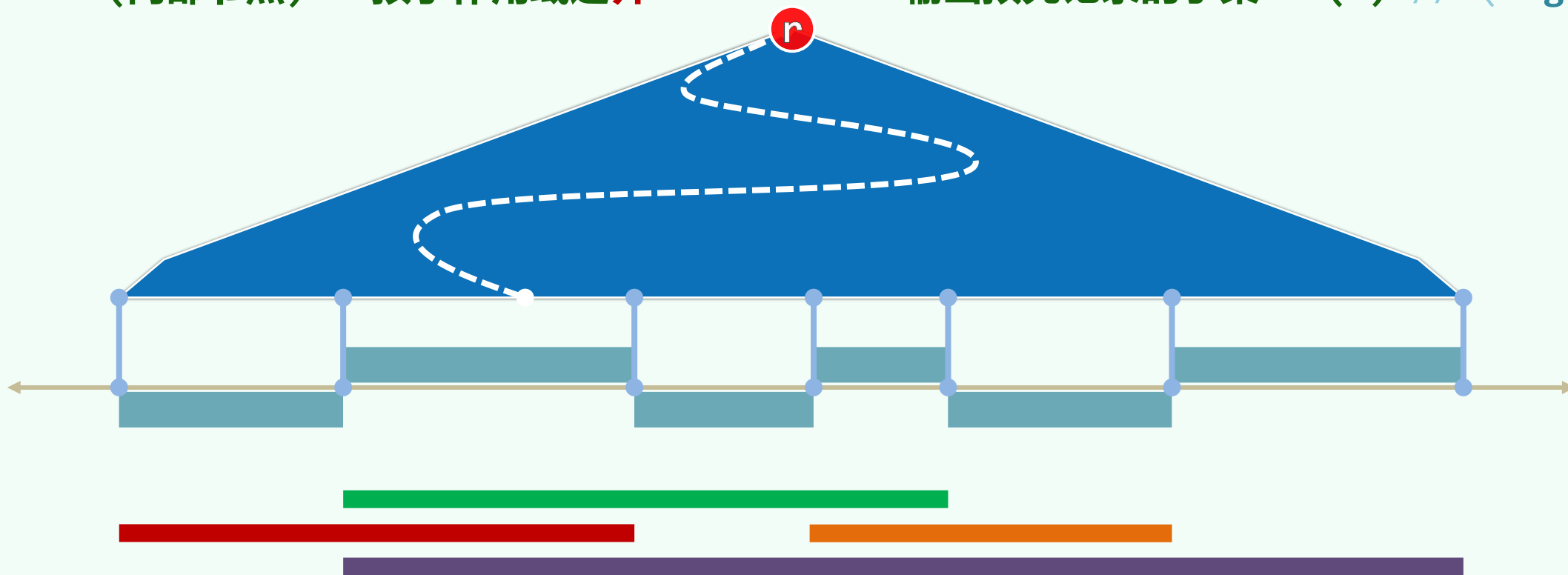
将有序向量，升级为BBST

❖ 作用域 $R(v)$

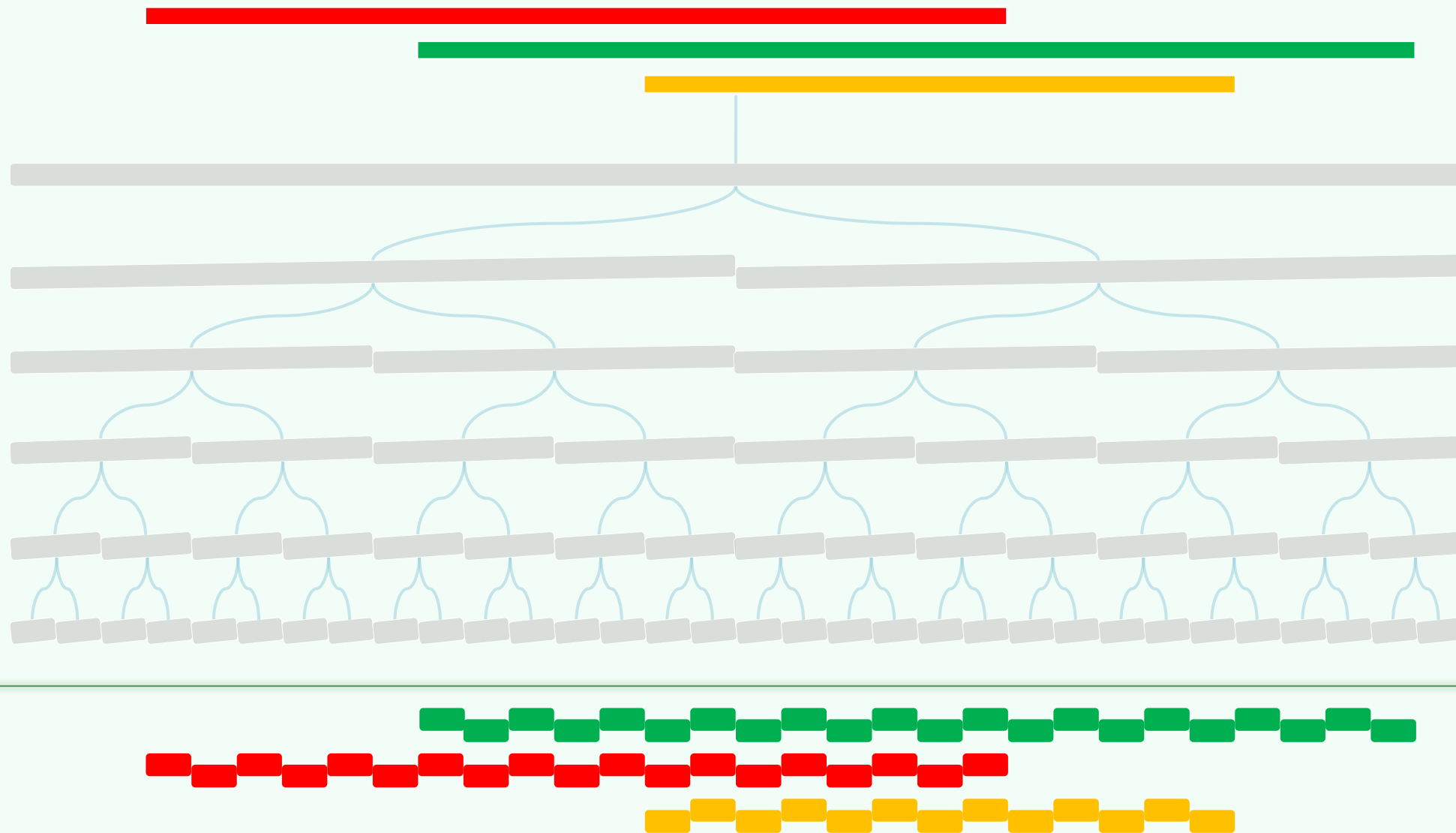
- $R(\text{叶节点}) = \text{对应的EI}$
- $R(\text{内部节点}) = \text{孩子作用域之并}$

❖ 叶子 v 预先记录 $\text{Int}(v) = \text{横跨}R(v)$ 的所有区间

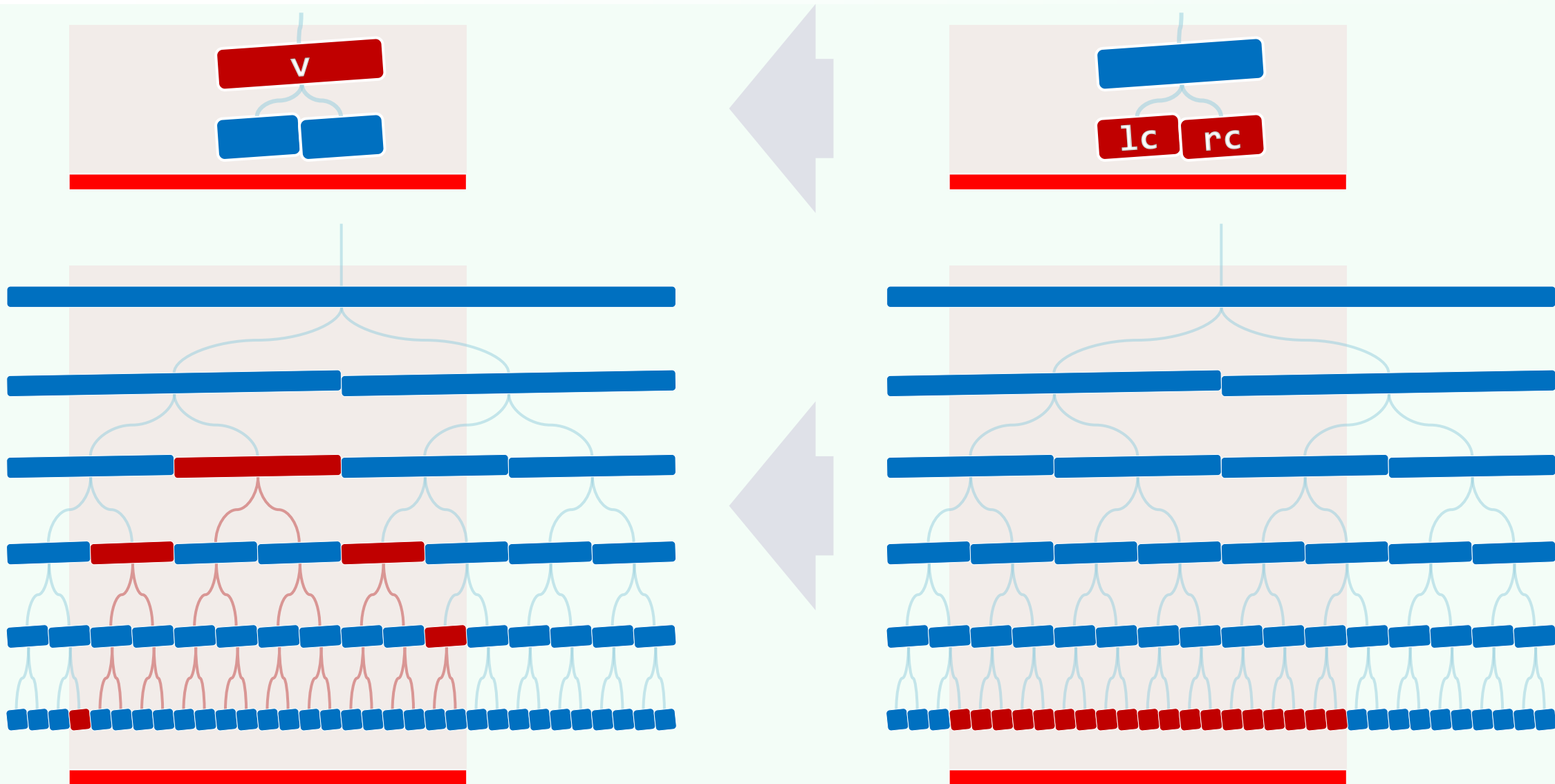
- ❖ 通过 $\text{BBST}::\text{search}(q_x)$ 找到对应的叶子 v ，即可输出预先记录的子集 $\text{Int}(v)$ $// O(\log n + r)$



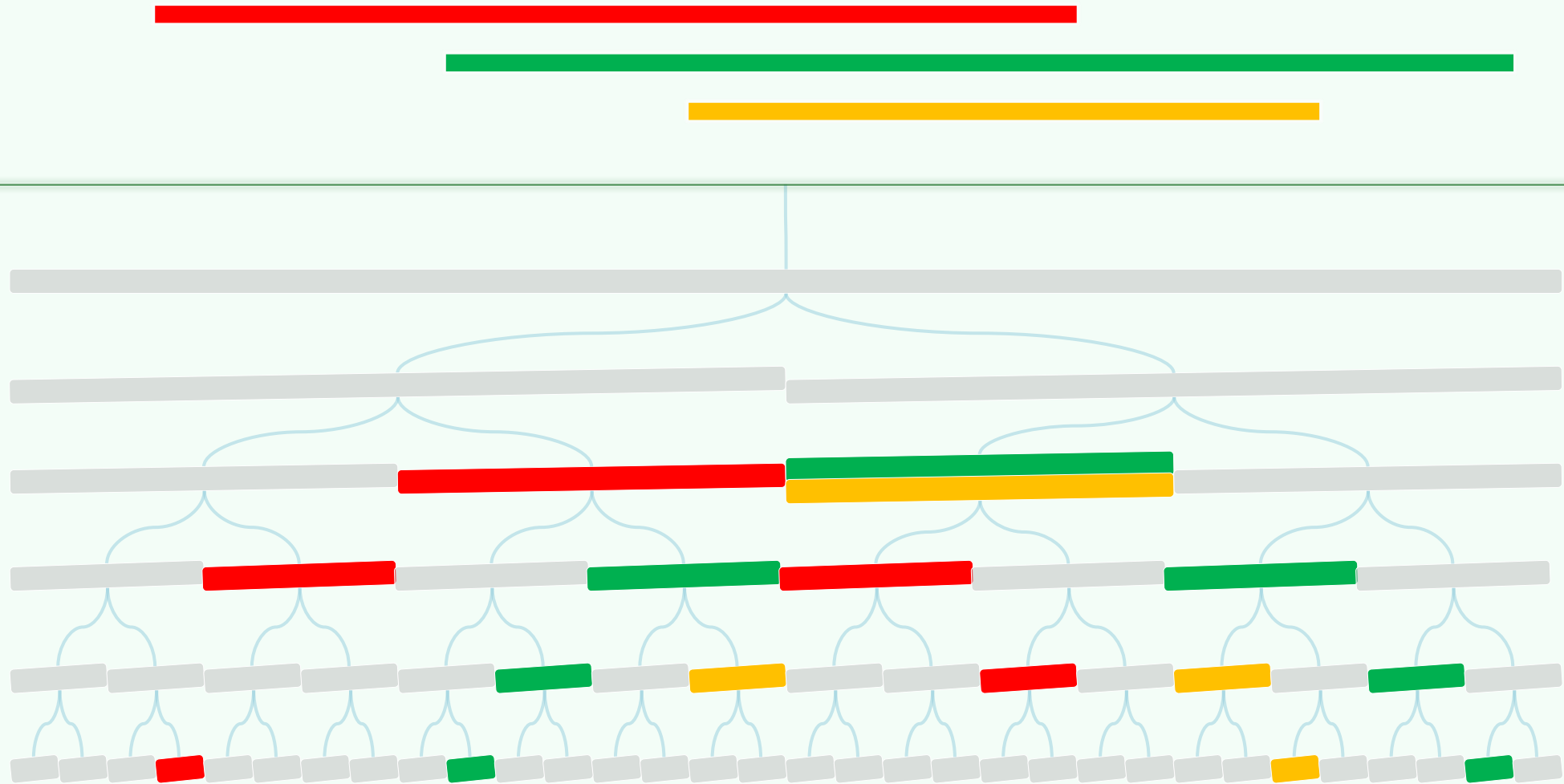
然而，最坏情况下...仍需... $\Omega(n^2)$



通过贪心地**合并**，每段区间至多只需记录 $O(\log n)$ 次



从而，空间复杂度降至 $O(n \log n)$



构造算法: buildSegTree(S)

对s中所有区间的端点**排序** $//O(n\log n)$

以所有EI为叶节点

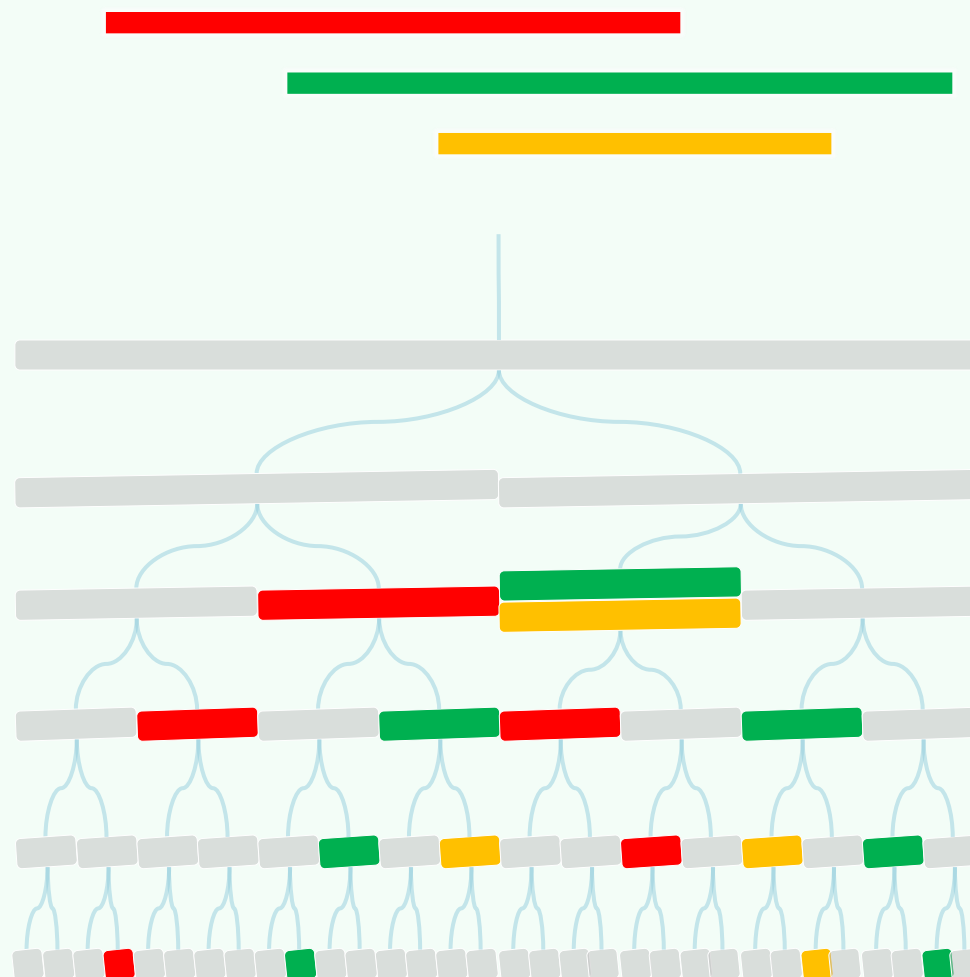
创建一棵完全二叉树T $//O(n)$

自底而上、逐层合并，确定

每个节点v的作用域**R(v)** $//O(n)$

for (S中的每段区间**s**)

 insertSeg(T.root, **s**)



构造算法: insertSeg(v, s)

```
if (  $R(v) \subseteq s$  ) //s覆盖了v之作用域
```

将s存入节点v

```
else
```

```
if (  $R(lc(v)) \cap s \neq \emptyset$  ) //recurse
```

```
insertSeg( lc(v), s )
```

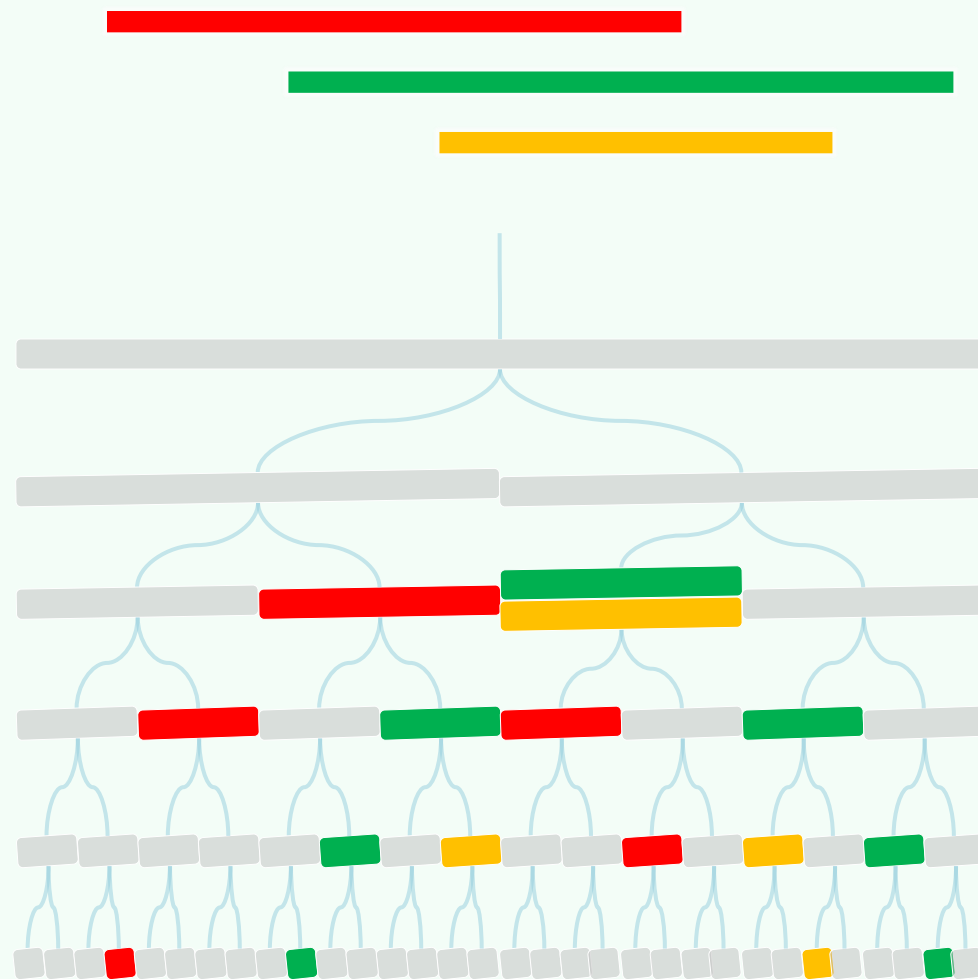
```
if (  $R(rc(v)) \cap s \neq \emptyset$  ) //recurse
```

```
insertSeg( rc(v), s )
```

❖ 在树中的每一层，至多访问4个节点

(2个存放s的复本，另2个递归)

故每段区间s的插入，只需 $O(\log n)$ 时间



查询算法: $\text{querySegTree}(v, q_x)$

报告 v 中记录的所有区间

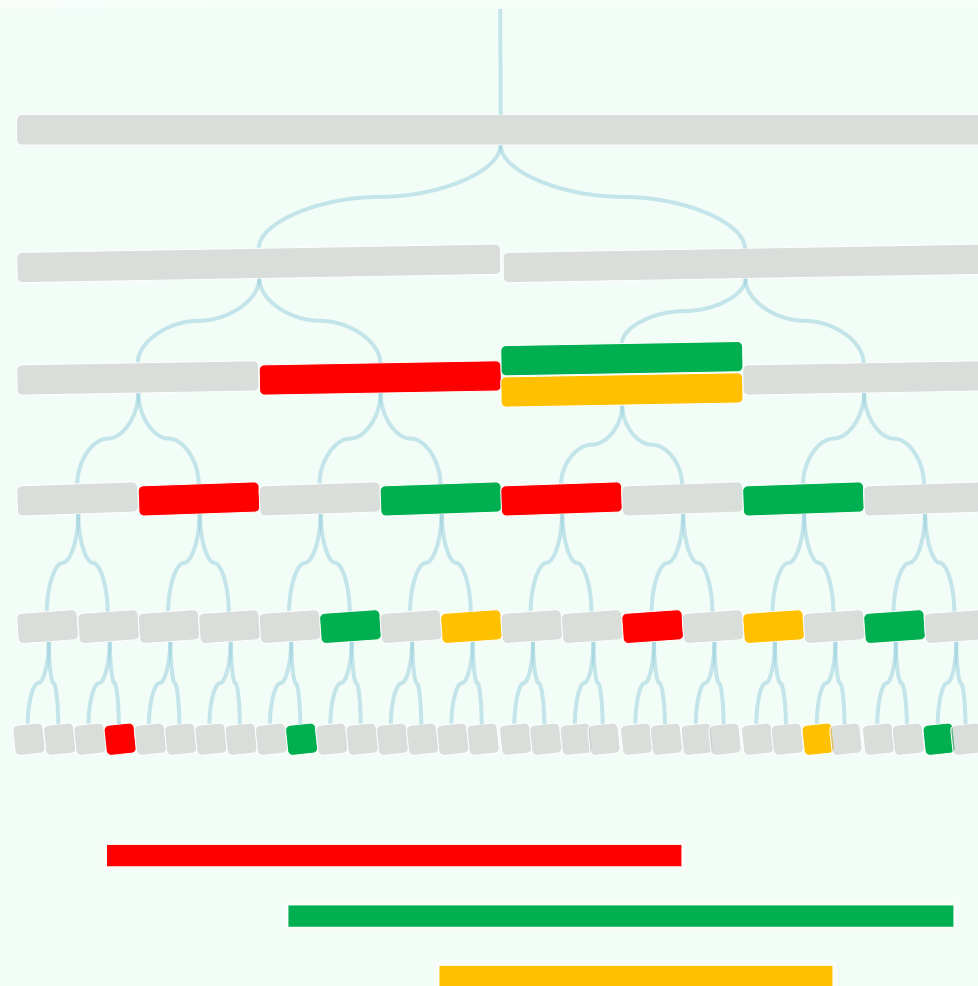
if (v 不是叶节点)

if ($q_x \in R(\text{lc}(v))$)

$\text{querySegTree}(\text{lc}(v), q_x)$

else // $q_x \in R(\text{rc}(v))$

$\text{querySegTree}(\text{rc}(v), q_x)$



查询时间: $\mathcal{O}(r + \log n)$

❖ 树中的每层仅需访问一个节点

累计 $\mathcal{O}(\log n)$ 个

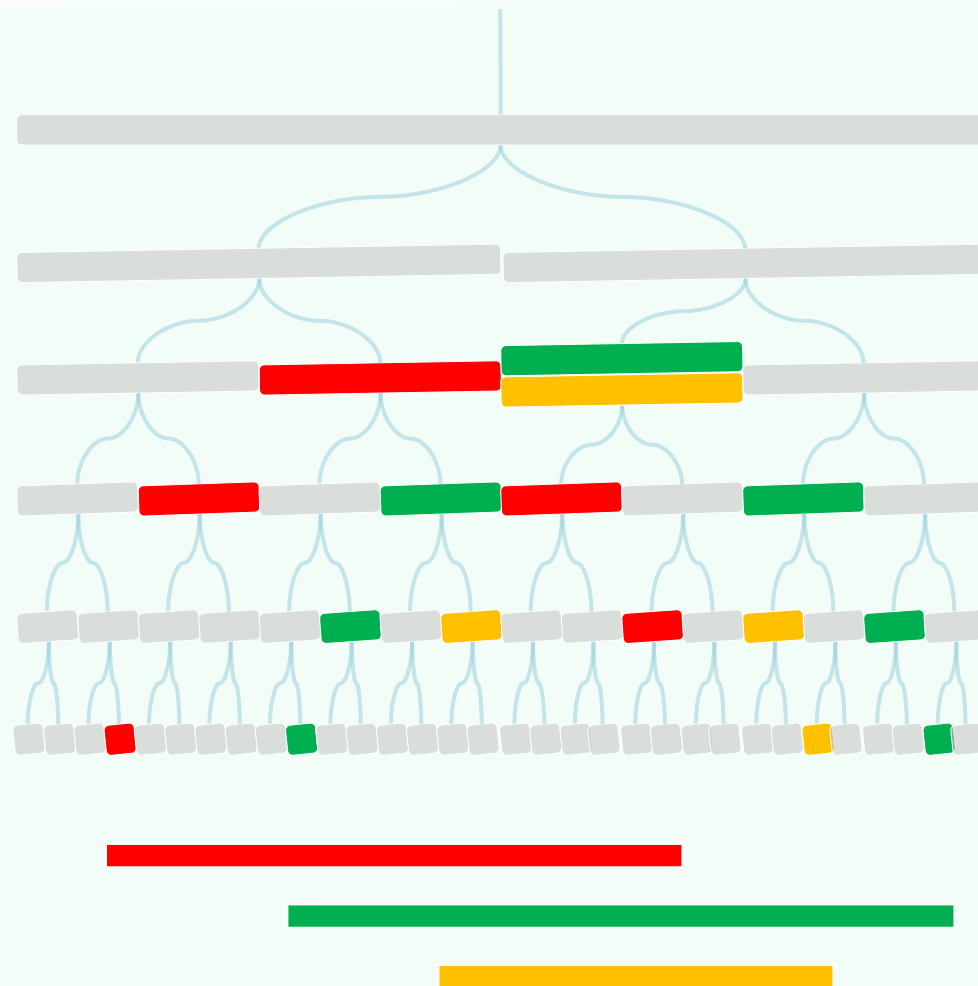
❖ 在每个节点 v 处

为输出 $\text{Int}(v)$ 所需的时间仅为

$$1 + |\text{Int}(v)| = \mathcal{O}(1 + r_v)$$

❖ 因此, 为输出所有命中区间

仅需 $\mathcal{O}(r)$ 时间

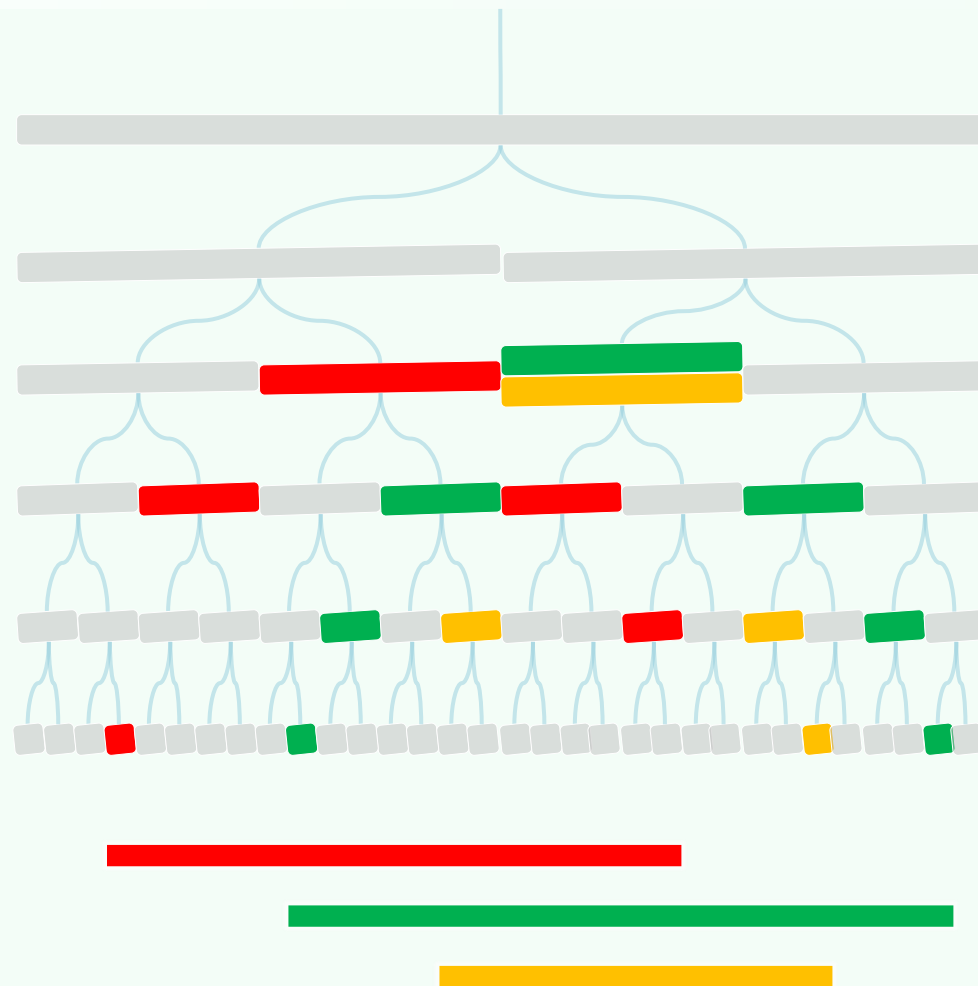


结论

❖ 任给x轴上的n段区间

- 可以在 $\mathcal{O}(n \log n)$ 时间内构造出
- 一棵占用 $\mathcal{O}(n \log n)$ 空间的线段树
- 借助它，每次穿刺查询都能

在 $\mathcal{O}(r + \log n)$ 时间内完成



$\theta > -C_1$

搜索树应用

范围树：范围查询

你这个人太敏感了。这个社会什么都需要，唯独不需要敏感

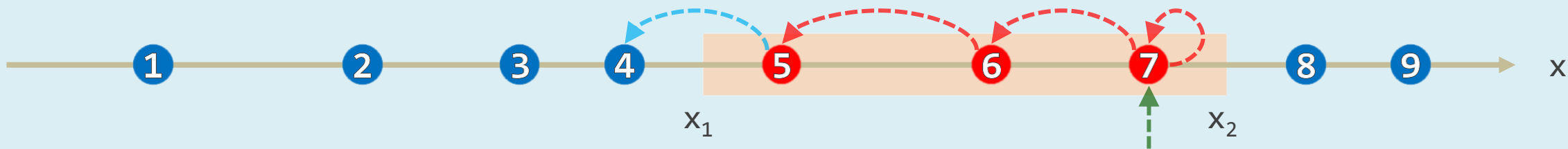
昔者明王必盡知天下良士之名：既知其名，又知其數；既知其數，又知其所在

邓俊辉

deng@tsinghua.edu.cn

1D Range Query = 有序向量 $\times$ (二分查找 + 遍历)

❖ x轴上**给定** n 个点 $P = \{p_1, p_2, p_3, \dots, p_n\}$; **任给**区间 $I = (x_1, x_2]$, P 中哪些点落在其中?



❖ 蛮力算法? 还是算了吧

❖ [output sensitive]

不能无差别地投入 $\mathcal{O}(n)$ 时间

❖ [Online] 既然 P 已**固定**

何不**预处理**为适当的数据结构? 比如...

❖ **有序向量** (添加哨兵 $p_0 = -\infty$)

❖ 每次查询仅耗时 $\mathcal{O}(1 + r + \log n)$

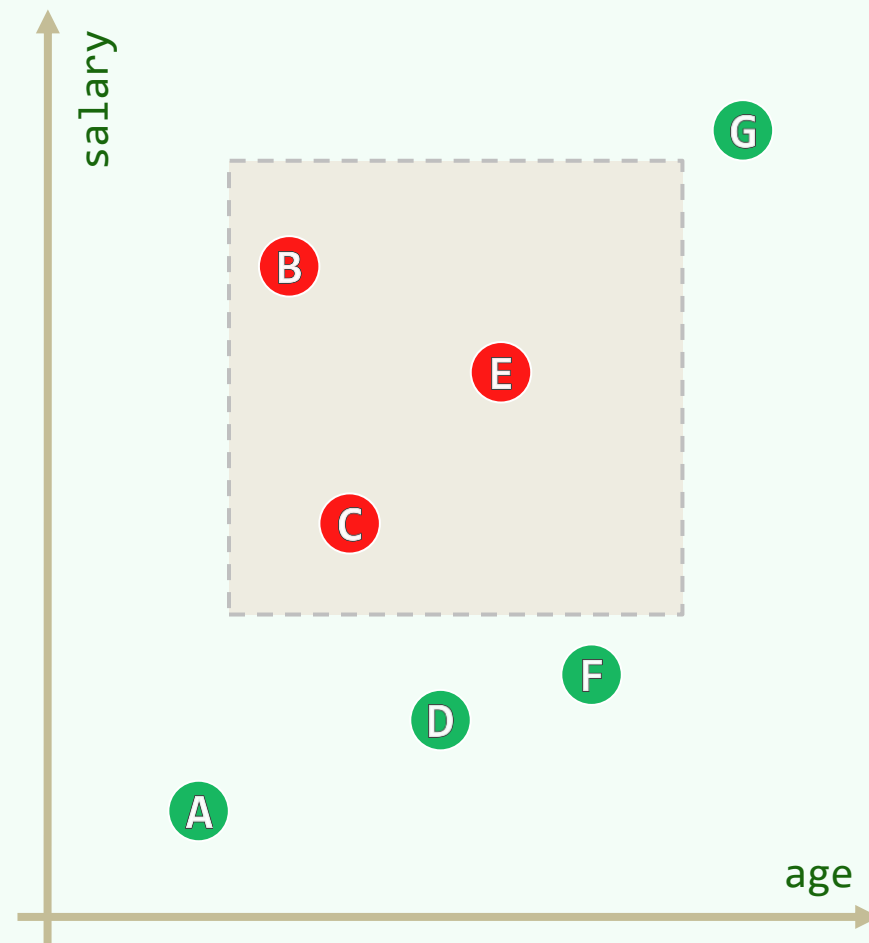
- **二分查找** $t = \text{search}(x_2) = \max\{i \mid p_i \leq x_2\}$

- 从 p_t 出发, 自后向前**遍历**; 直至于 p_s 出界

❖ 很可惜, 该策略不容易拓展到**二维**

Planar Range Query

- ❖ 给定平面点集 $P = \{ p_1, p_2, p_3, \dots, p_n \}$
- ❖ 对任何矩形区域 $R = (x_1, x_2] \times (y_1, y_2]$
报告落在区域内的所有点，即 $R \cap P$
- ❖ 各点之间并无有意义的次序
二分查找对此无能为力
- ❖ 容斥原理：形式优雅，思路简明，速度也不错，只是
空间消耗太大，无法承受
- ❖ 还是，诉诸直觉吧...



两阶段，分步走：思路

❖ 2维查询 = 2次的1维查询

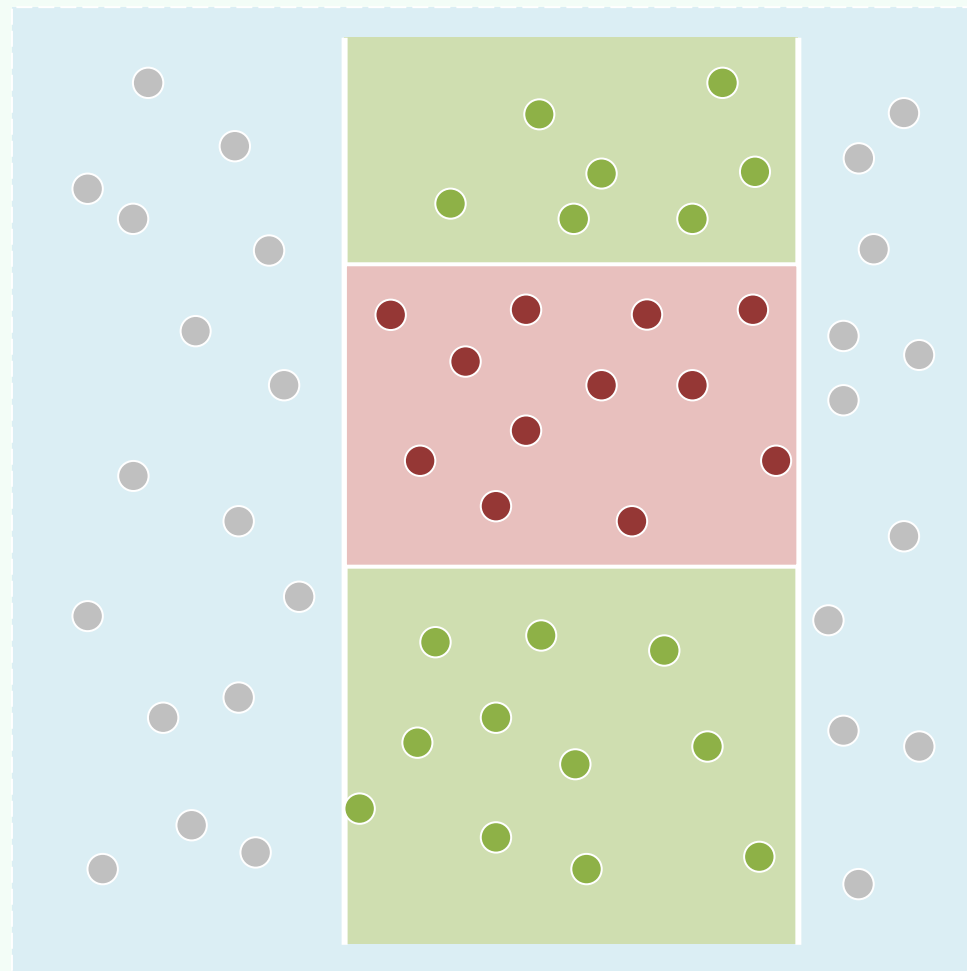
- x-查询：先筛出 $(x_1, x_2]$ 内的点，进而
- y-查询：再筛出 $(y_1, y_2]$ 内的点

总体耗时 = $\mathcal{O}(r_1 + r + \log n)$

❖ 推而广之，每一次m维的正交范围查询

都可通过m次的1维正交查询来回答

❖ 事情，果然就如此...简单？



两阶段，分步走：最坏情况

❖ 完全有可能...

- x-查询命中了几乎所有点 // $r_1 = \Theta(n)$
- y-查询排除了几乎所有点 // $r = \Theta(1)$

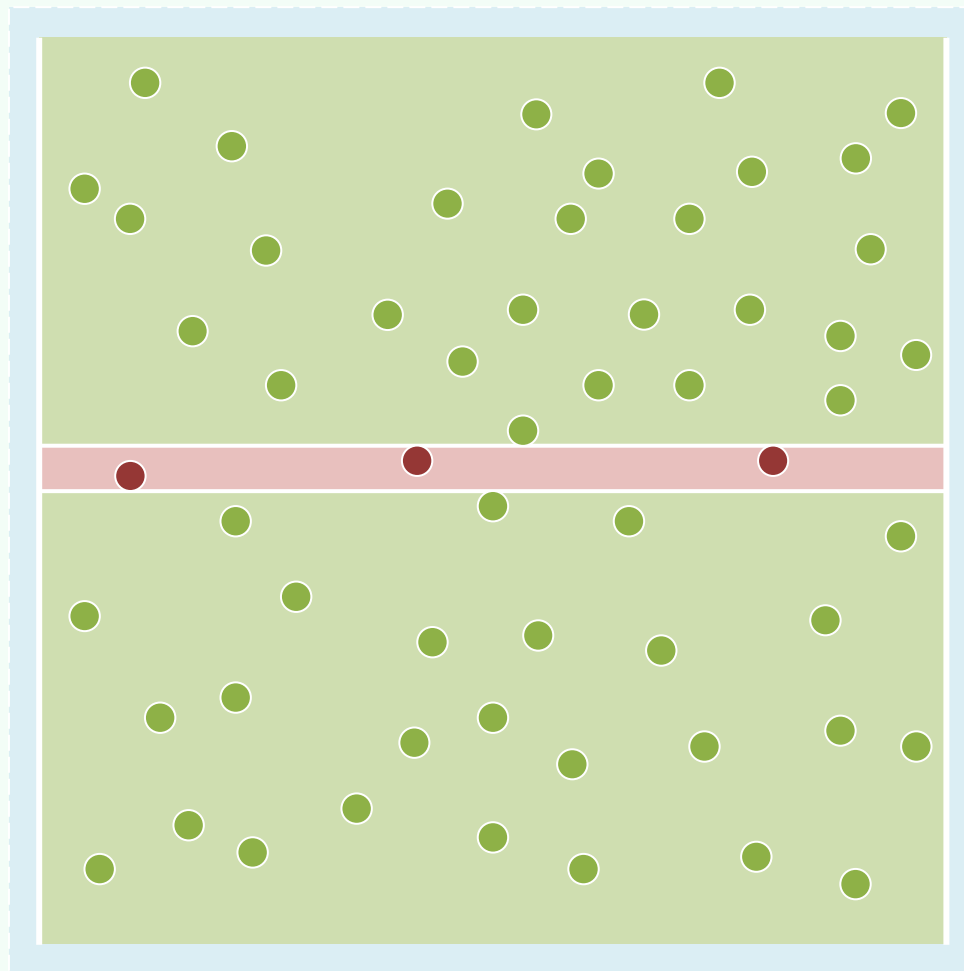
❖ 总体耗时 = $\mathcal{O}(r_1 + r + \log n) = \mathcal{O}(n)$

还是没能实现输出敏感

❖ 不要怀疑两阶段的思路，其实它完全可行

当然，要有精巧的技术、强大的数据结构来支撑

❖ 作为热身，回到一维情况，看看BBST能有何作为...



07-C2

搜索树应用

范围树：BBST

邓俊辉

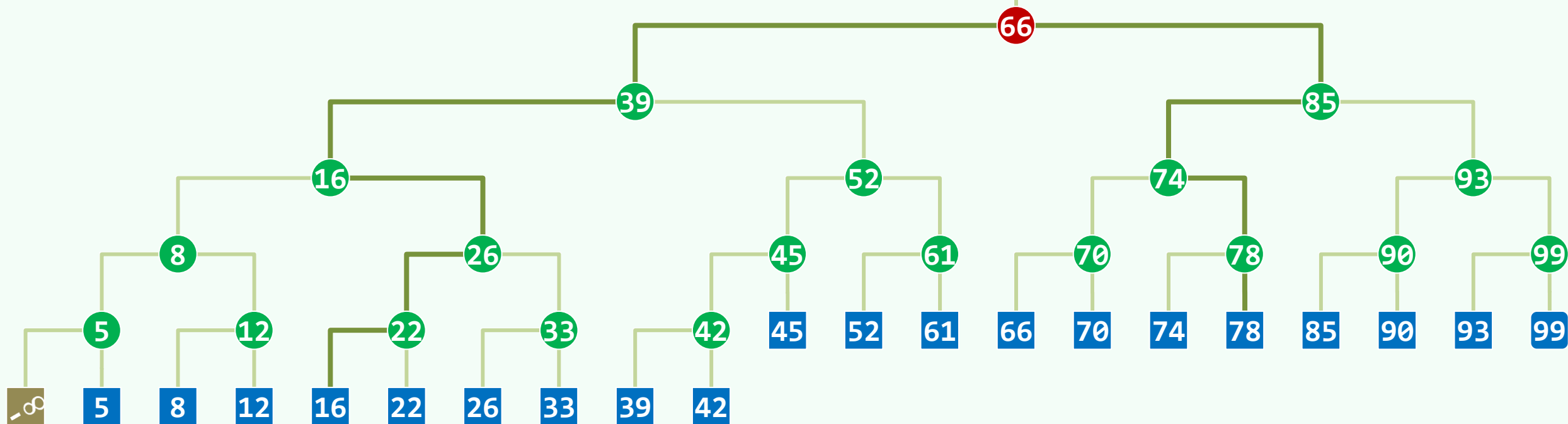
deng@tsinghua.edu.cn

只在此山中，雲深不知處

完全的BBST

$$\forall v, v.key = \min\{u.key \mid u \in v.rTree\} = v.succ.key$$

$$\forall u \in v.lTree, u.key < v.key \quad \forall u \in v.rTree, u.key \geq v.key$$



❖ $\text{search}(x)$ = 不超过 x 的最大者

两次搜索、两条藤蔓；一个祖先、 $\log n$ 棵子树

❖ `search(17) ~ vine(16)`

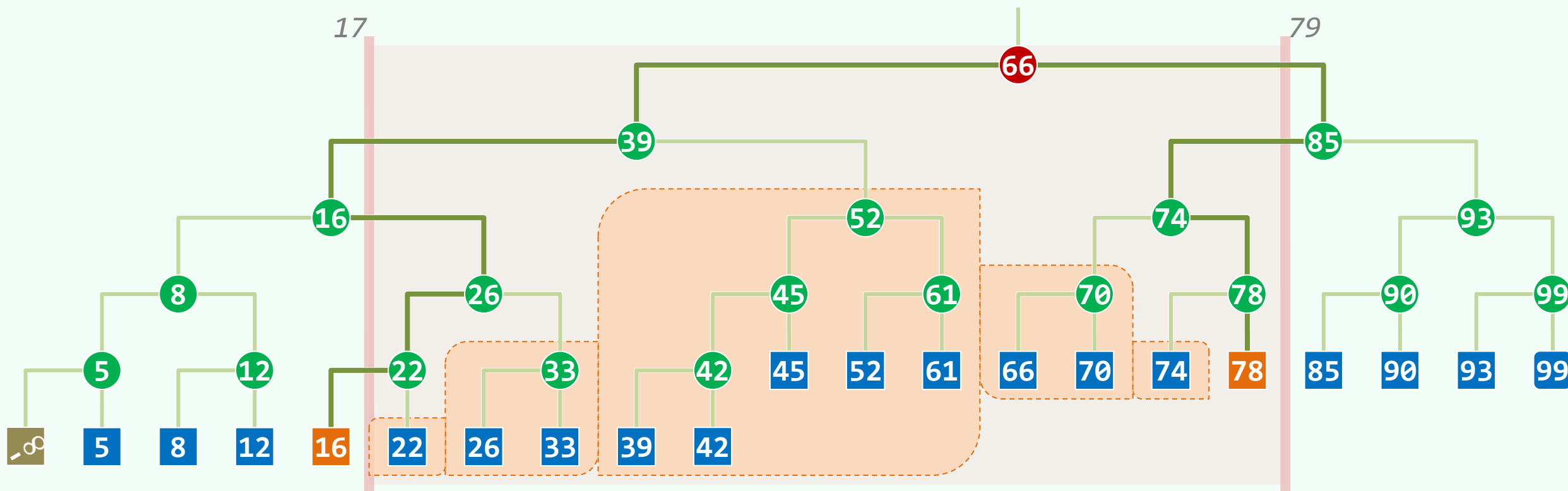
所有右子树

❖ Lowest Common Ancestor

LCA(16, 78) = 66

❖ `search(79) ~ vine(78)`

所有左子树



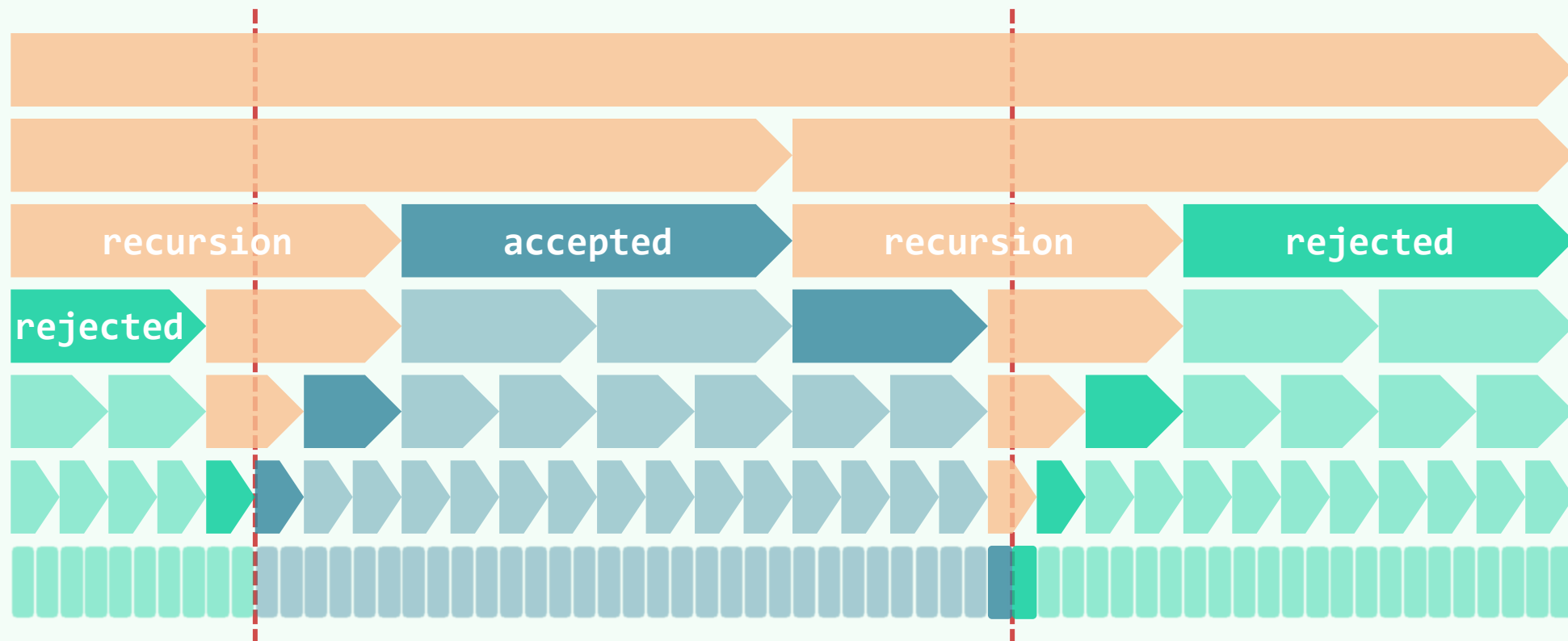
查询: $\mathcal{O}(\log n)$

预处理: $\mathcal{O}(n \log n)$

存储: $\mathcal{O}(n)$

热刀来切 $O(\log n)$ 层的巧克力蛋糕

- ❖ $\text{Region}(u)$ 被 $\text{Region}(v)$ **包含**，当且仅当节点 u 是 v 的**后代**
- ❖ $\text{Region}(u)$ 与 $\text{Region}(v)$ **无交**，当且仅当 u 和 v 不是**直系血缘关系**



- ❖ 所谓的查询，无非是将节点分为**三类**：accepted + rejected + recursion



搜索树应用

范围树：多层搜索树

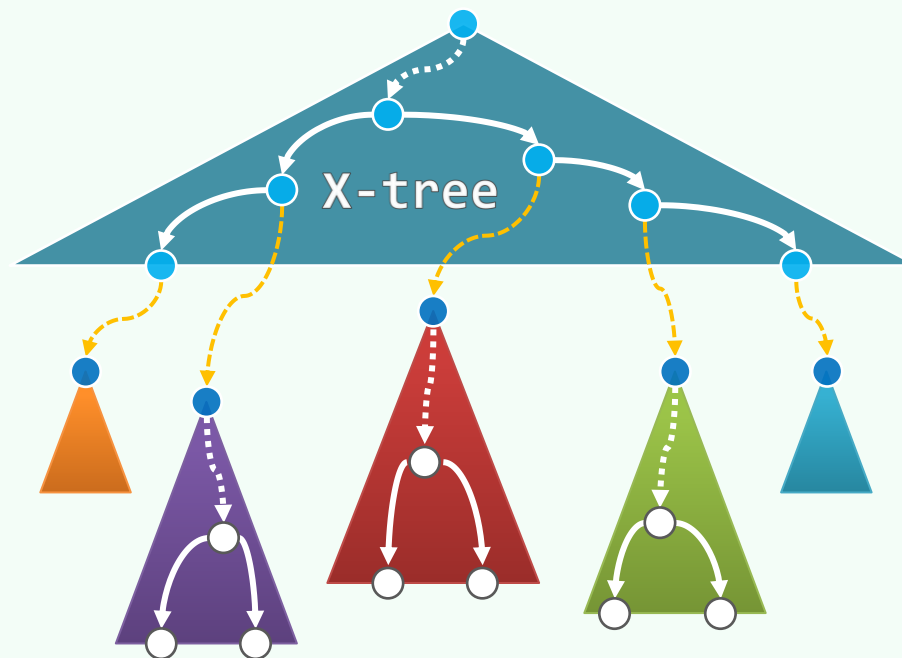
几株不知名的树，已脱下了黄叶
只有那两三片，多么可怜在枝上抖怯
它们感到秋来到，要与世间离别

邓俊辉

deng@tsinghua.edu.cn

两阶段，分步走：x-Tree * y-Trees

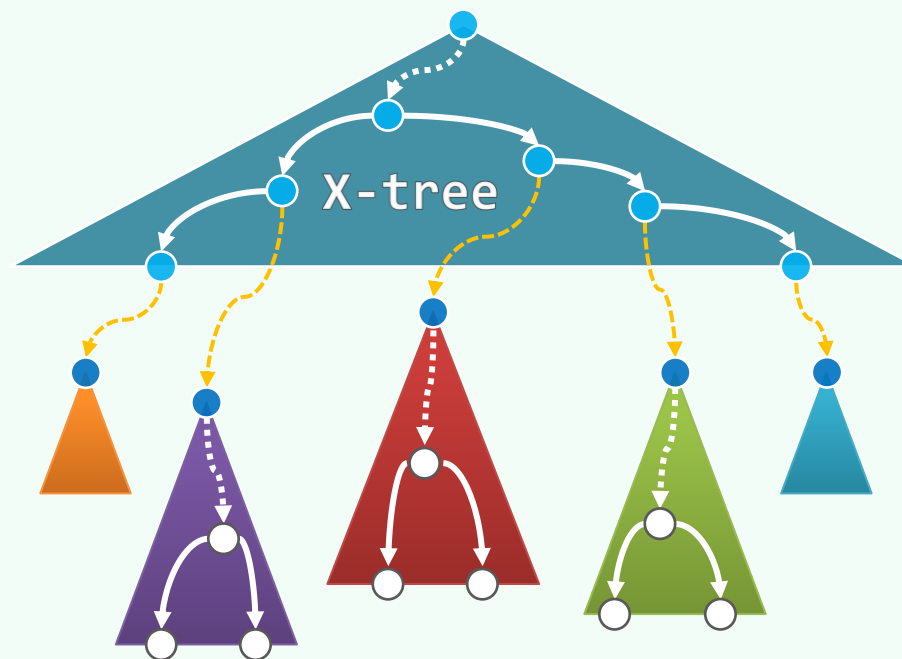
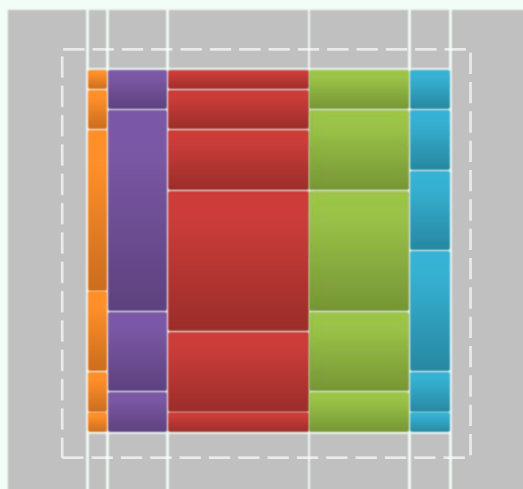
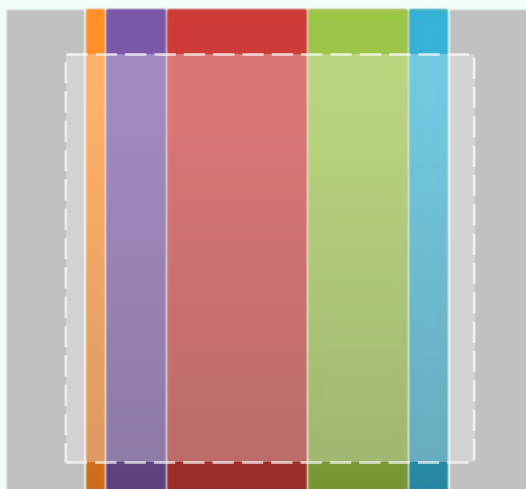
- ❖ 针对x-查询，构建一棵x-树
 - ❖ 对于x-树中的任何一棵子树v
 - 构造一棵关联的y-树，并
 - 建立**关联**：由v可**找到**对应的y-树
 - ❖ 如果还有更多维度，也可照此**逐层**推广
- 多层搜索树 | **M**ulti-**L**evel **S**earch **T**ree
- ❖ 借助MLST，如何高效完成范围查询？
- 以下以**2维**为例...



查询: x-Query * y-Query

❖ 经过x-query(x_1, x_2), 在x-树中**锁定** $\mathcal{O}(\log n)$ 棵子树 //不要急于输出! 不要输出! 不要输出!

仅需 $\mathcal{O}(\cancel{r} + \log n)$ 时间



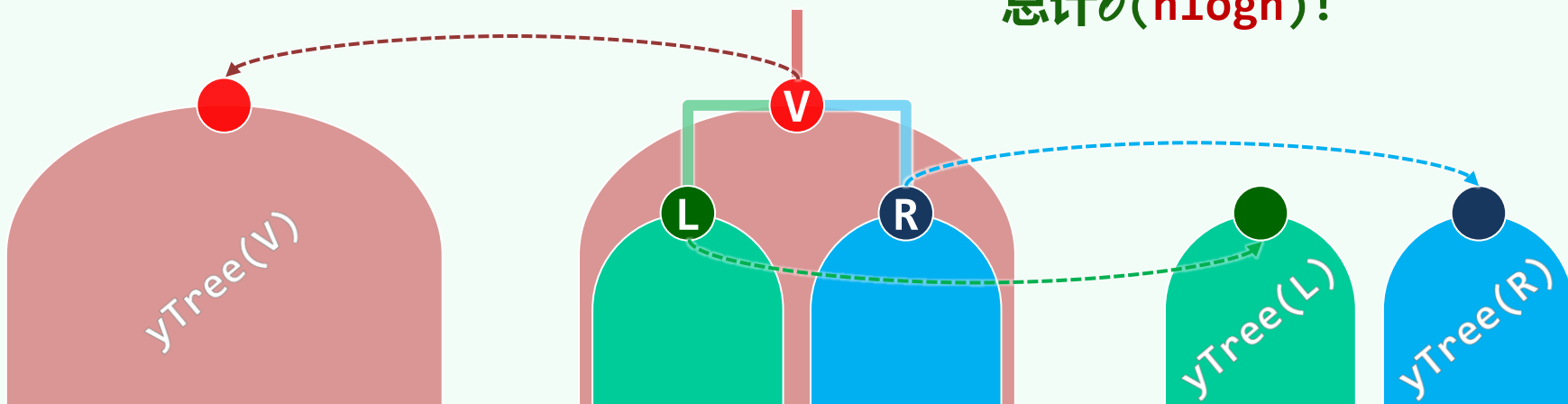
❖ 找到关联的 $\mathcal{O}(\log n)$ 棵y-树

分别经过y-query(y_1, y_2), 查找并**输出**命中的点

$$\text{总体耗时} = \sum_{i=1}^{\mathcal{O}(\log n)} [r_i + \mathcal{O}(\log n)] = \mathcal{O}(r + \log^2 n)$$

预处理：自底而上、逐层合并

- ❖ 关键看y-树：各自独立地构建，太费时间
- ❖ 从叶子出发，通过逐层地**捉对合并**
自底而上地**逐渐**构造出x-树
- ❖ 在此过程中，y-树也**同步**地捉对合并
由小而大、由矮而高地**逐个**生成
- ❖ 设**子树** $X(1)$ 与 $X(r)$ 刚被建成；
关联的 $Y(1)$ 与 $Y(r)$ 也是
- ❖ 构造 $X(p)$ 只需 $O(1)$ 时间
构造 $Y(p)$ 只需 $O(n)$ ！
- ❖ **同高度**的所有y-树，只需 $O(n)$ 时间
总计 $O(n \log n)$ ！



存储空间

❖ x-树只占 $\mathcal{O}(n)$ 空间，关键还是看y-树

- 共计 $\mathcal{O}(n)$ 棵，只怕会爆炸吧？
- 还好，合计也不过 $\mathcal{O}(n \log n)$

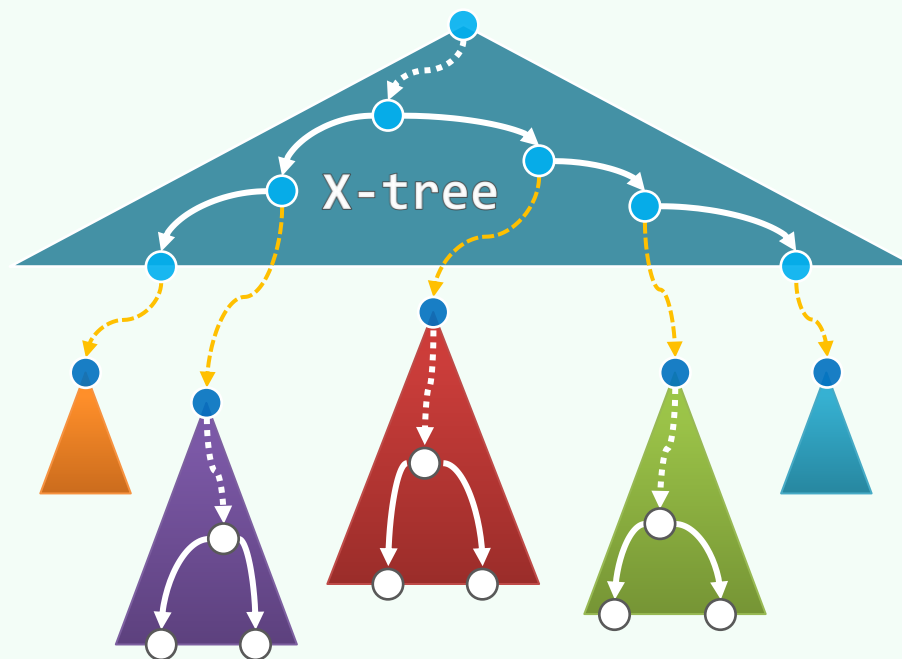
❖ 两种统计口径，殊途同归

- 从待查询点的视角来看

每个点都记录在 $\mathcal{O}(\log n)$ 棵y-树中

- 从x-树的视角来看

每一层节点所关联的y-树，互无重叠，总计 $\mathcal{O}(n)$



更高维度

❖ 由欧氏空间 $\mathcal{E}^d$ 中的任意 n 个点，都可以

A> 在 $\mathcal{O}(n \cdot \log^{d-1} n)$ 时间内

构造出一棵 d 层的MLST

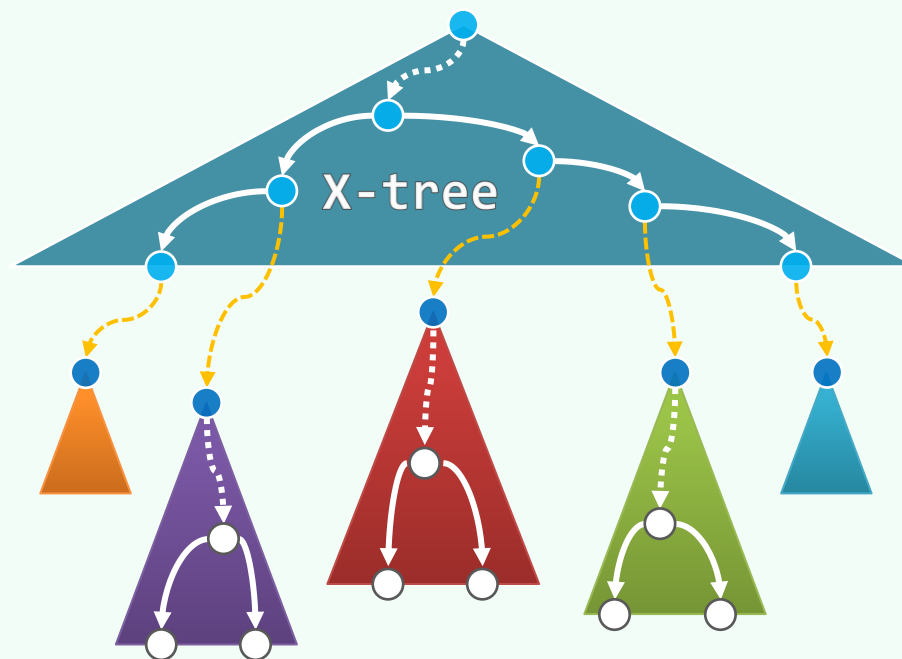
B> 该结构只需 $\mathcal{O}(n \cdot \log^{d-1} n)$ 空间

C> 借助该结构，每次 d 维范围查询

都可以在 $\mathcal{O}(r + \log^d n)$ 时间内完成

❖ 借助**分散层叠**的技巧，将MLST升级为**范围树** |

即可将C>改进至 $\mathcal{O}(r + \log^{d-1} n) \dots$



07-C4

搜索树应用

范围树：分散级联

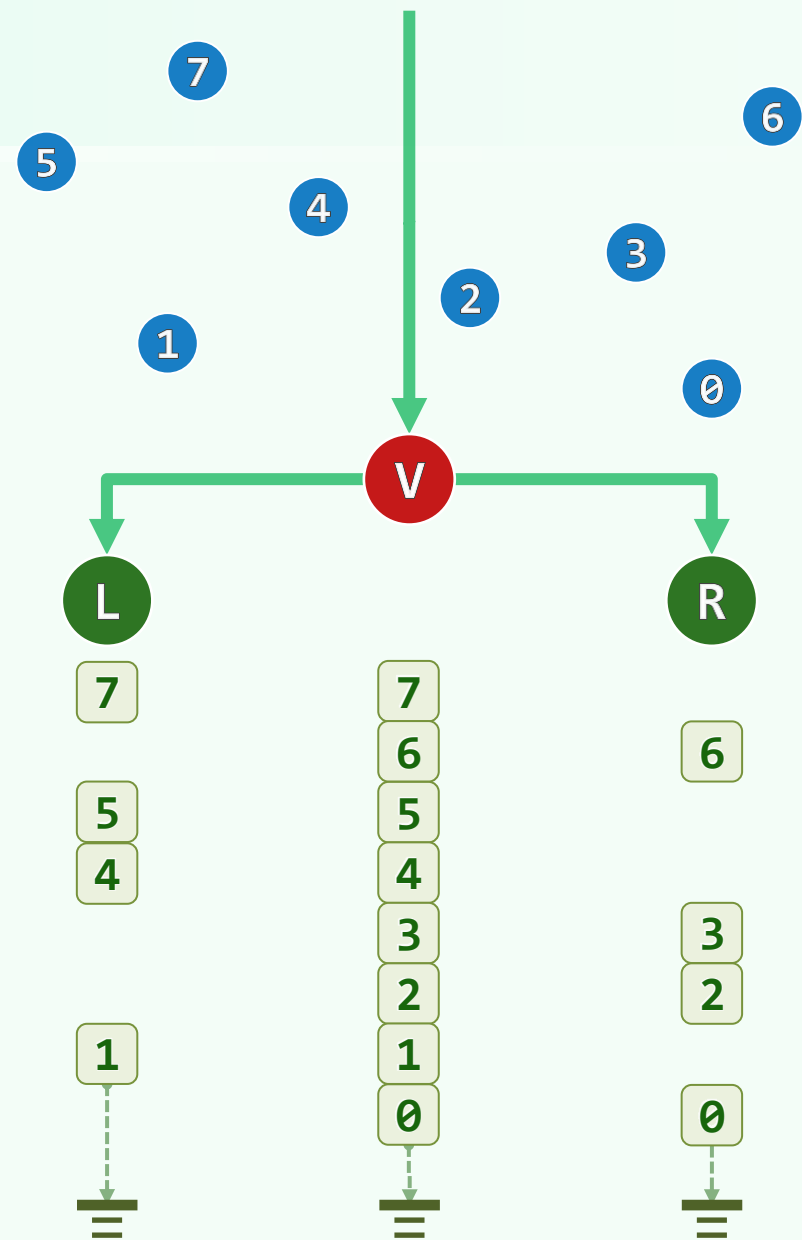
邓俊辉

deng@tsinghua.edu.cn

顺藤摸瓜

BBST<BBST<T>> --> BBST<List<T>>

- ❖ 属于**最后**一个阶段的y-查询，与此前阶段的查询不同
不需继续筛拣，可以直接输出
- ❖ 既如此，完全不必将其组织为一棵BBST
实际上，简化为一个**有序列表**即足矣
- ❖ 以**二维**为例：最后的阶段是y-查询
- ❖ 可将整个结构改造为
 - 一棵**x-树**，其中
 - 每个节点都配有一个关联的**y-列表**



相关性 | Coherence

❖ 时间成本无非两个部分

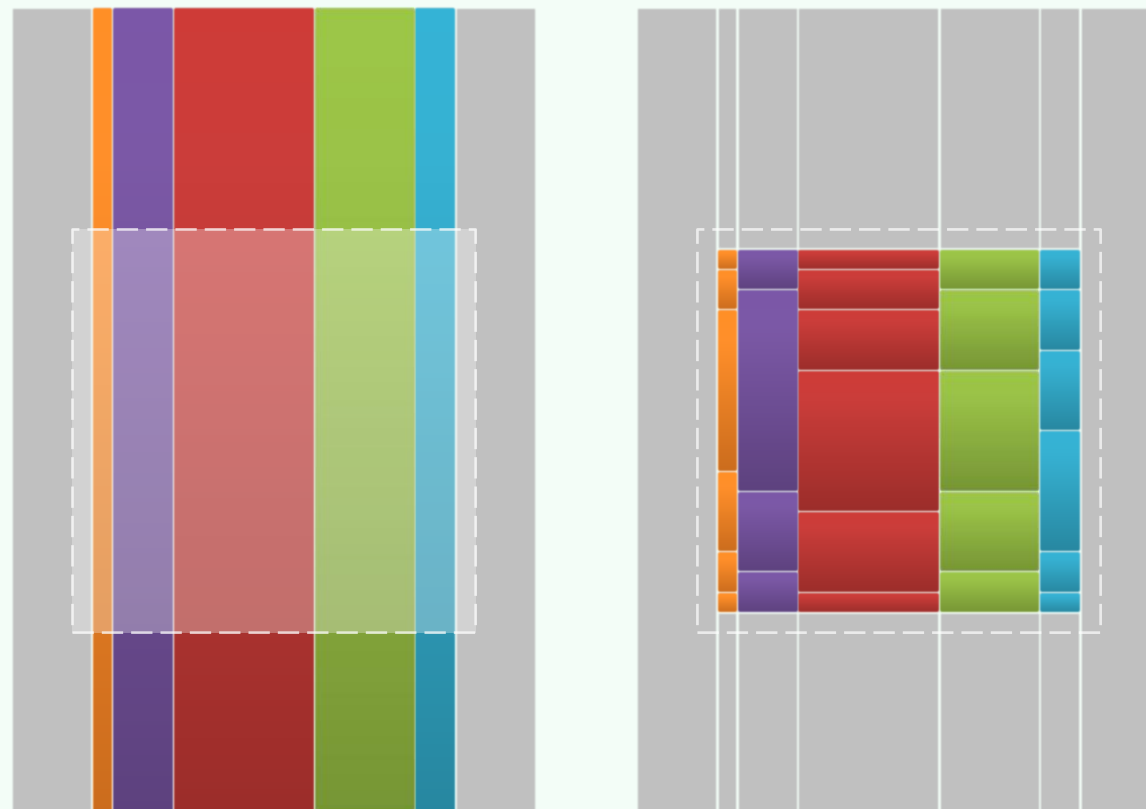
- 输出 $\mathcal{O}(r)$: 属于**本分**，已无改进可能
- 查找 $\mathcal{O}(\log^2 n)$: 似乎也**紧**，其实不然

❖ 共有 $\mathcal{O}(\log n)$ 次**y**-query(y_1, y_2)

- 虽然针对的**y**-树各自不同
- 但对应的**范围**却完全相同

❖ 同一批的这些**y**-查询

- 既然如此紧密地**相关**
- 为何却要孤军奋战、**各行其是**?



分散级联: Shortcuts

❖ Fractional Cascading

在父、子的y-列表之间, 建立**捷径引用**

- $yList(L) \leftarrow yList(V) \rightarrow yList(R)$

- 由 $V.search(y)$, 可以**直接**

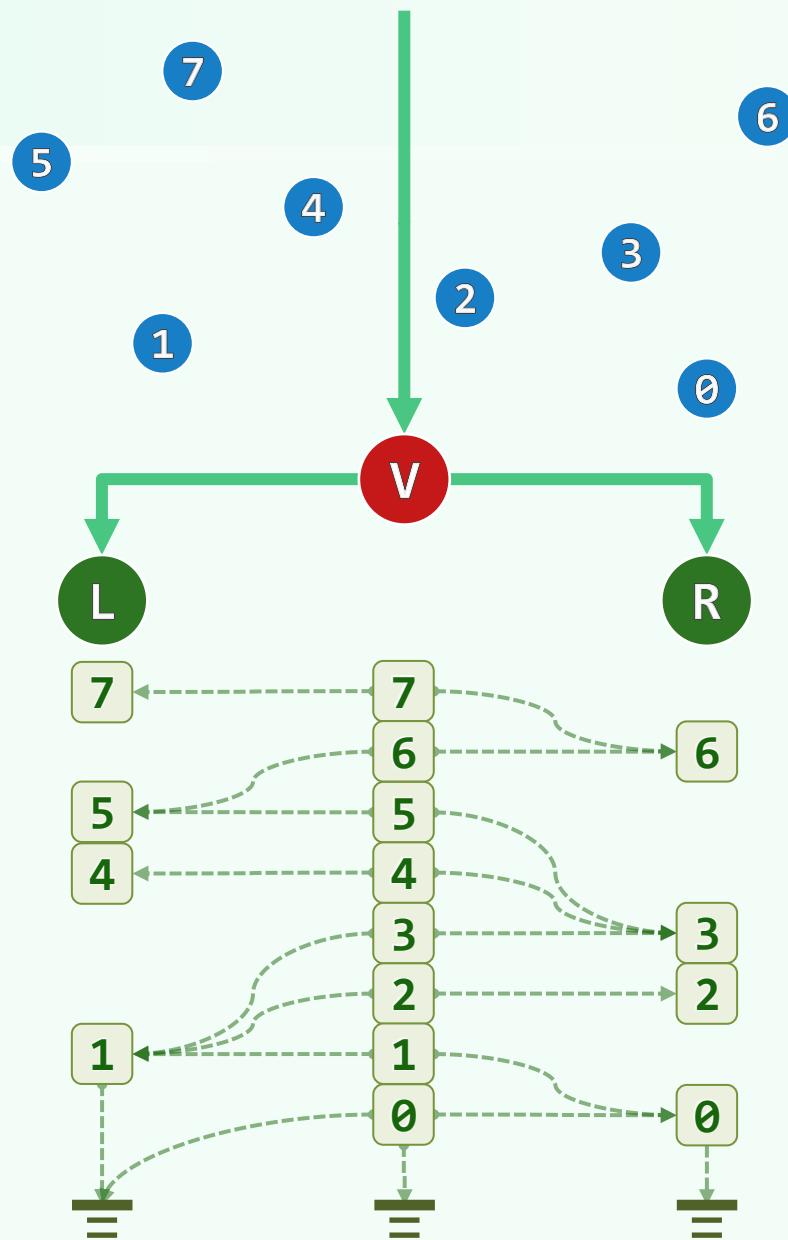
得到 $L.search(y)$ 和 $R.search(y)$

❖ 也就是说, 在y-查询阶段

- 一旦花过 $O(\log n)$ 时间完成了 $LCA.query()$

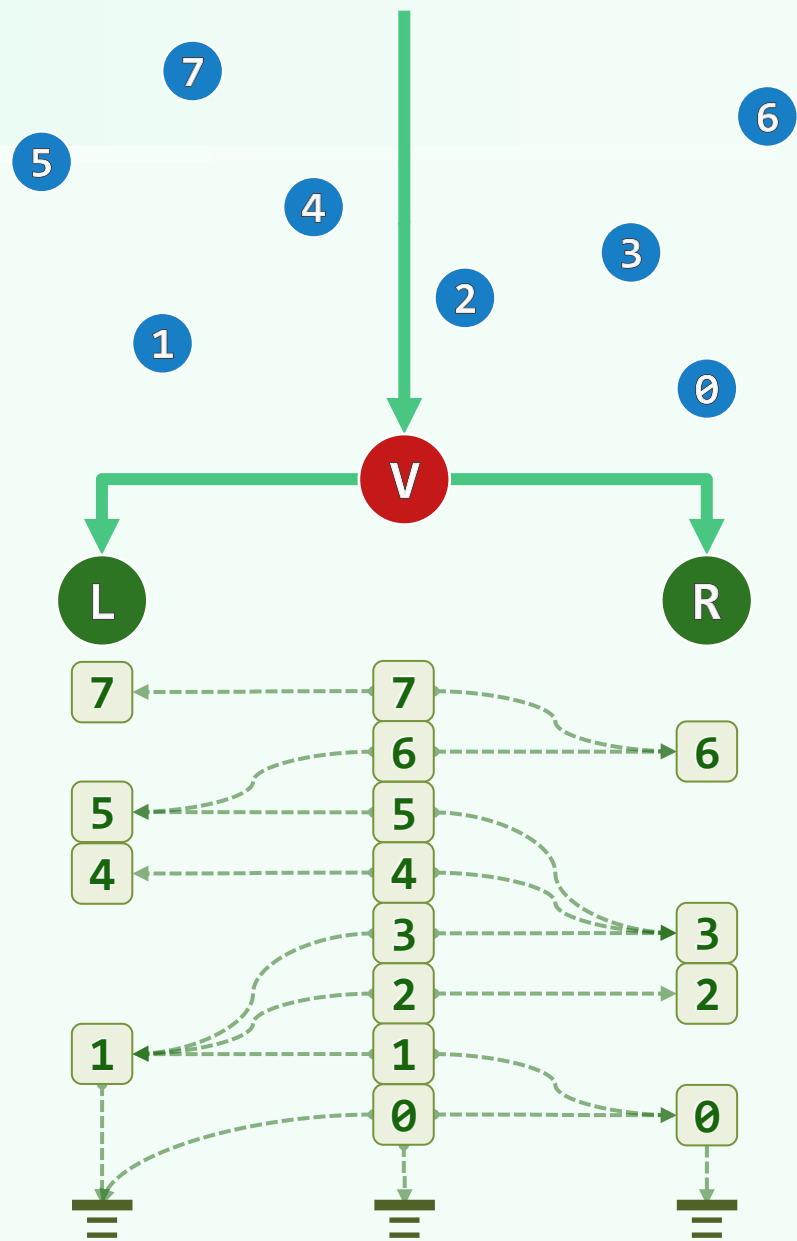
- 后代们便可**相跟着**完成y-查询

各自仅需 $O(1)$ 时间



分散级联：二路归并

- ❖ 就内容而言： $vList(V) = vList(L) + vList(R)$
- ❖ 如此前所推荐地，所有 y -列表应当紧随着 x -树
自底而上地逐层合并而成
- ❖ $vList(V)$ 可由 $vList(L)$ 与 $vList(R)$
经二路归并而生成
- ❖ 任何点 p 在即将出列归入 $vList(V)$ 时
 $vList(L)$ 与 $vList(R)$ 的首元素
即是 p 之捷径应当指向的点
- ❖ 预处理的整体时间复杂度，并不会因此增加



范围树

❖ 引入了分散级联技术的MLST

称作**范围树** | range tree

❖ 由平面上的任意 n 个点，都可以

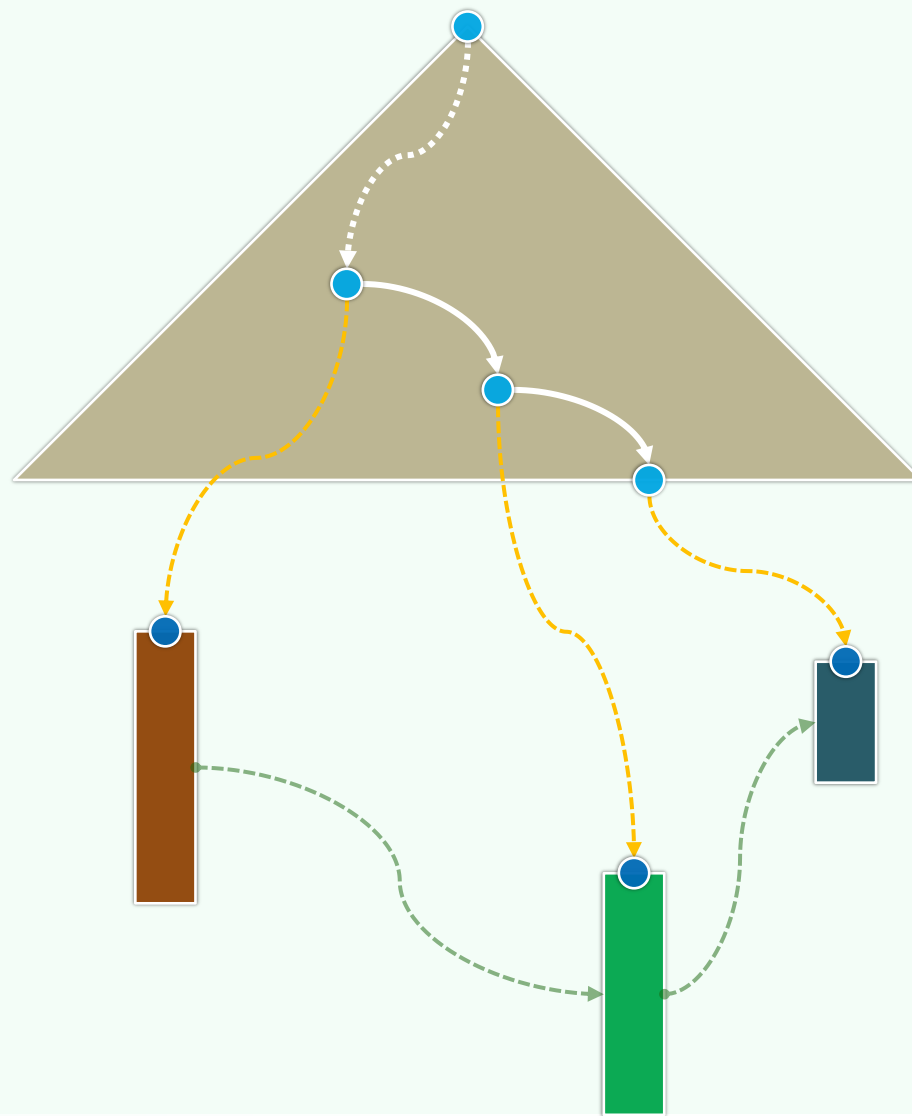
A> 在 $\mathcal{O}(n \cdot \log n)$ 时间内

构造出一棵2层的范围树

B> 该结构只需 $\mathcal{O}(n \cdot \log n)$ 空间

C> 借助该结构，每次二维范围查询

都可以在 $\mathcal{O}(r + \log n)$ 时间内完成



更高维度

❖ 遗憾的是，对MLST结构而言

分散级联的技巧只能运用于**最后**的维度

❖ 由欧氏空间 $\mathcal{E}^d$ ($d \geq 2$) 中的任意 n 个点，都可以

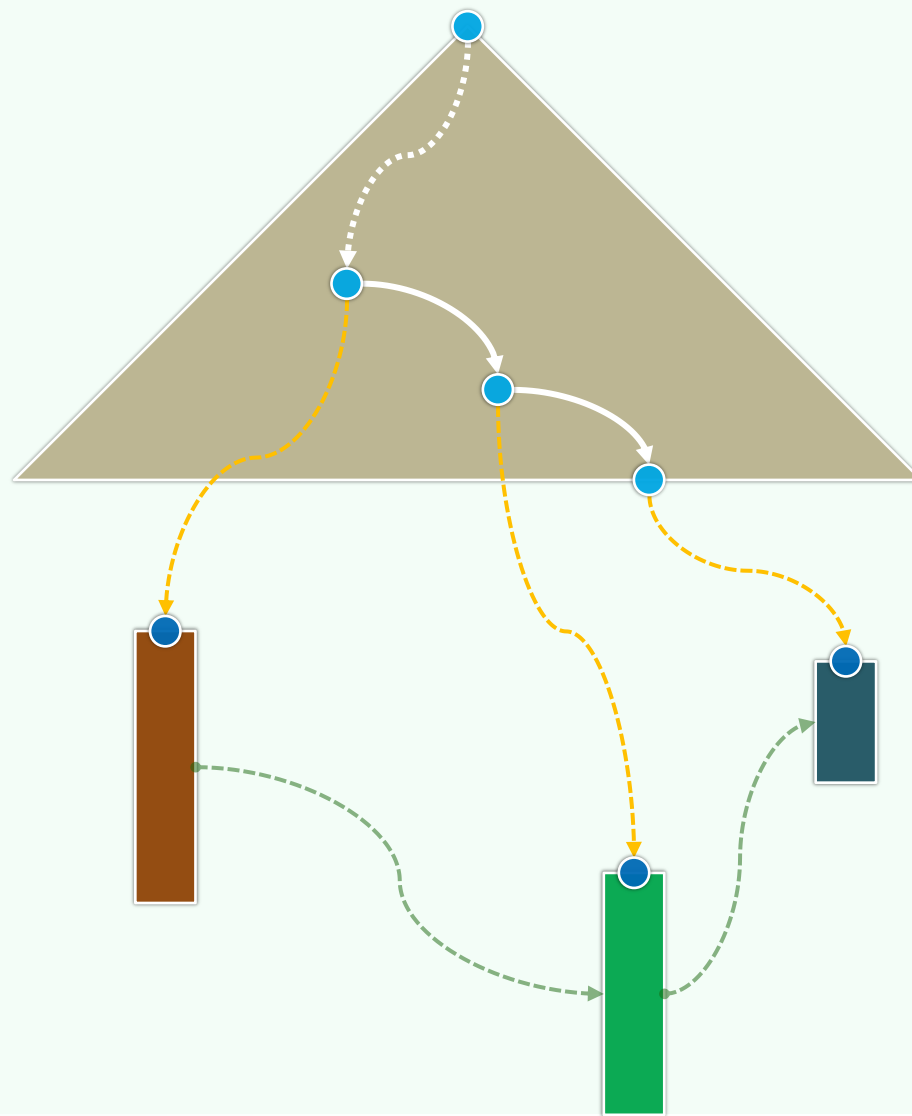
A> 在 $\mathcal{O}(n \cdot \log^{d-1} n)$ 时间内

构造出一棵 d 层的MLST

B> 该结构只需 $\mathcal{O}(n \cdot \log^{d-1} n)$ 空间

C> 借助该结构，每次 d 维范围查询

都可以在 $\mathcal{O}(r + \log^{d-1} n)$ 时间内完成



搜索树应用

kd-树：结构

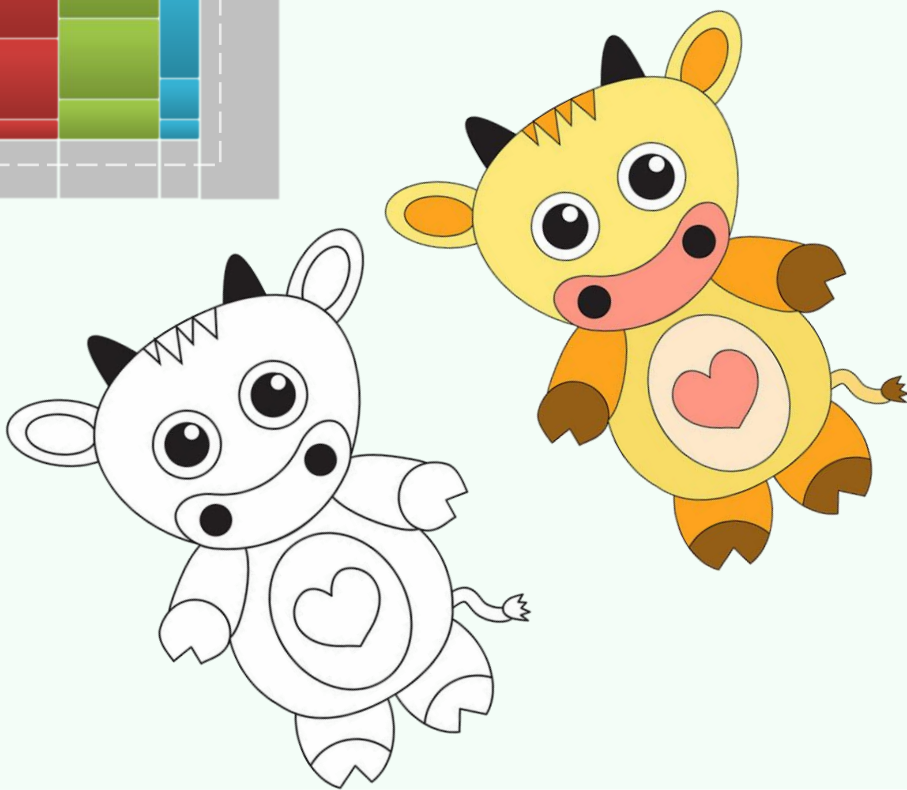
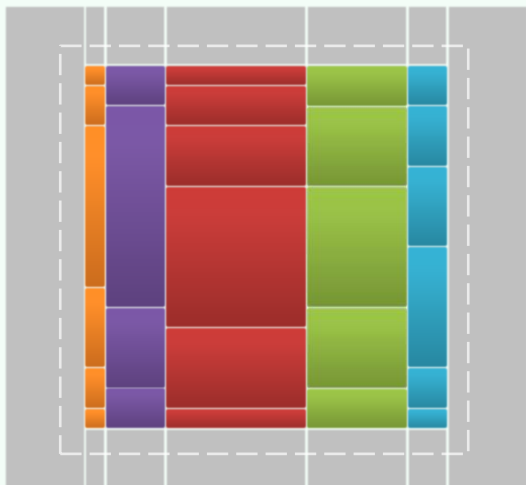
韦小宝跟著她走到桌边，只见桌上大白布上钉满了几千枚绣花针，几千块碎片已拼成一幅完整无缺的大地图，难得的是几千片碎皮拼在一起，既没多出一片，也没少了一片

夫仁政，必自經界始...經界既正，分田制祿可坐而定也

邓俊辉

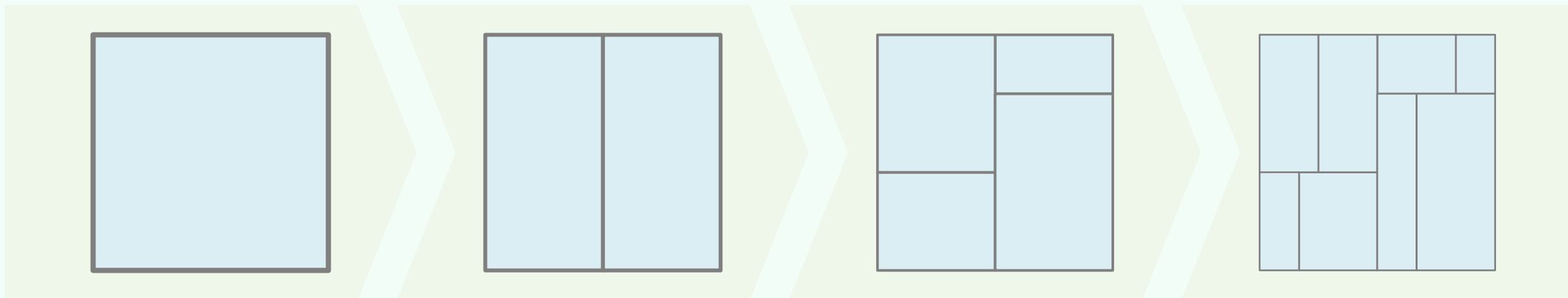
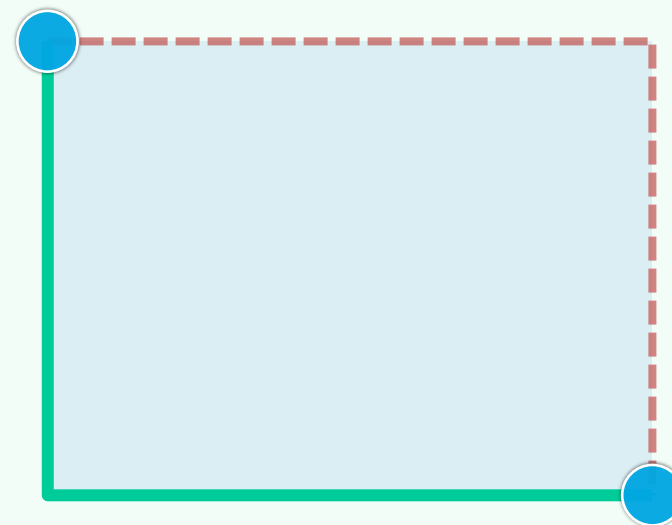
deng@tsinghua.edu.cn

由粗而细，逐步求精



思路

- ❖ 为利用BBST来支持二维的区域查询，可以递归地划分平面，并将分出来的矩形区域组织为一棵二叉树
- ❖ 首先，根节点对应于整个平面（足够大的包围矩形）
然后，交替地按x、y坐标划分，直至...
- ❖ 为避免歧义，可约定每个矩形区域都是左闭右开、下闭上开



构造：算法buildKdTree(P,d)

```
// Construct a 2d-tree for point set P at depth d
```

```
if (|P| < 2) return createLeaf(P) //base
```

```
Root = createKdNode()
```

```
Direction = even(d) ? VERTICAL : HORIZONTAL
```

```
Line = findMedian(P, root.SplitDirection) //O(n)!
```

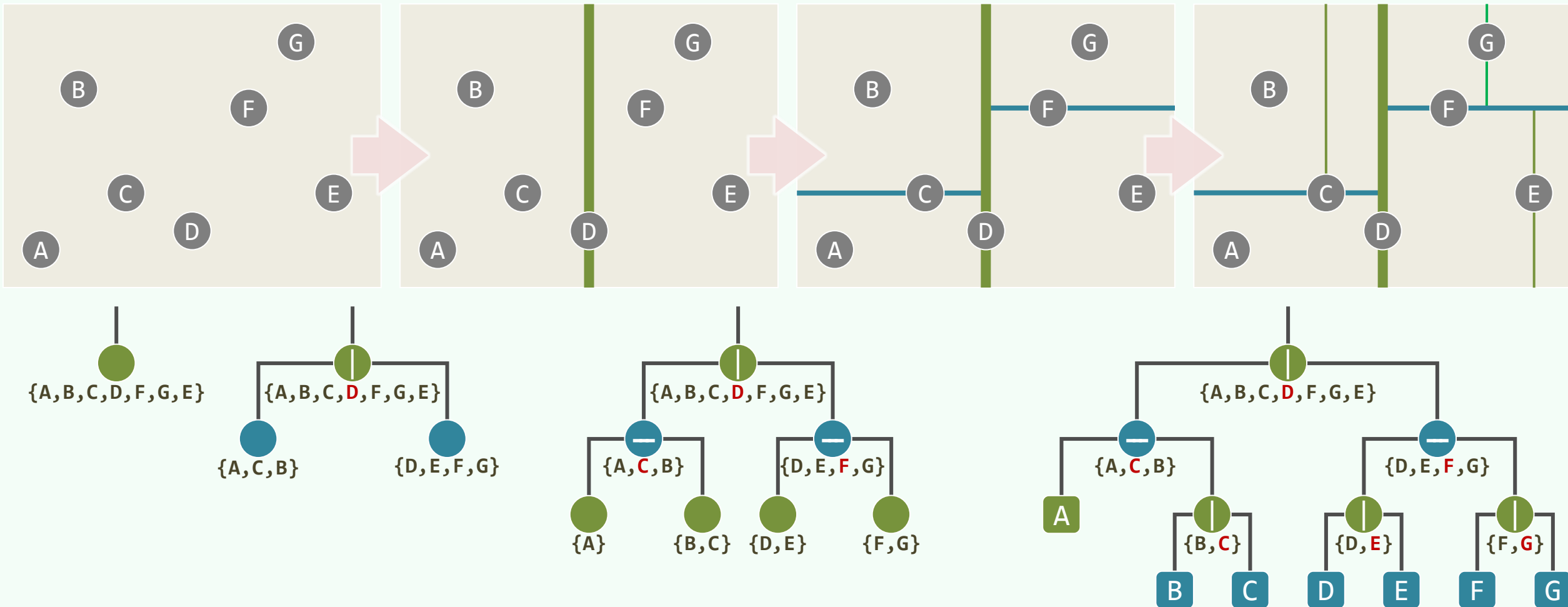
```
( L, R ) = divide( P, Direction, Line ) //DAC
```

```
Root.lc = buildKdTree(L, d+1) //recurse
```

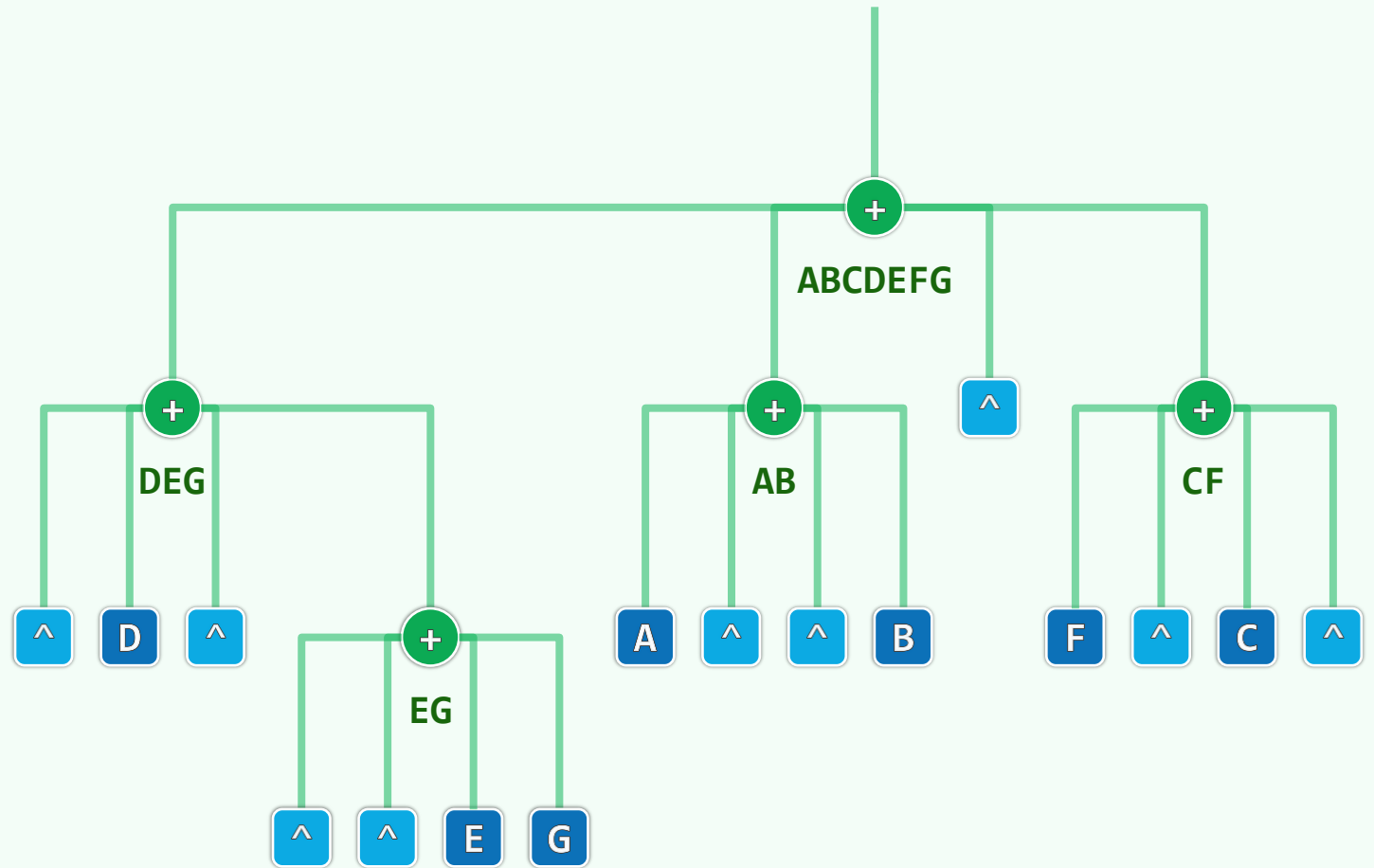
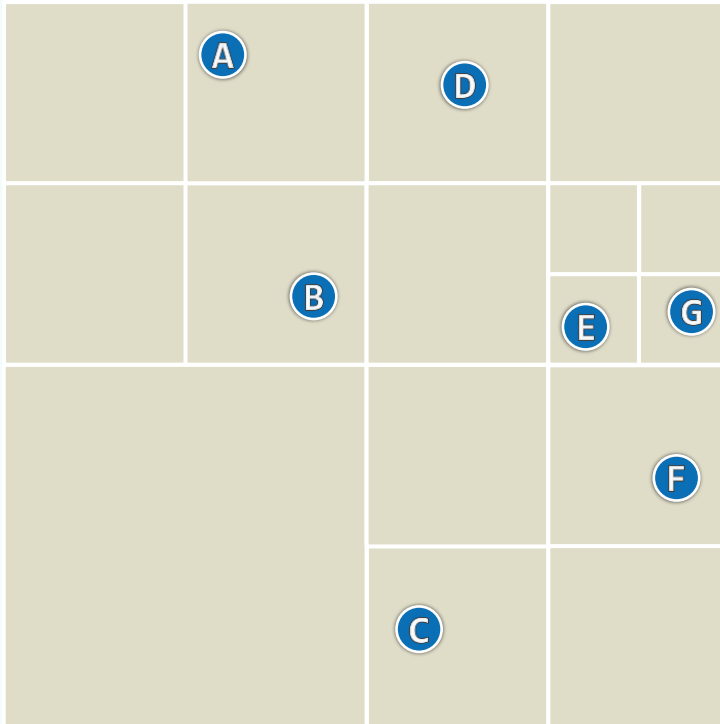
```
Root.rc = buildKdTree(R, d+1) //recurse
```

```
return Root
```

构造：实例



特例: Quadtree



性质

❖ 树中的每个**节点** v , 都对应于平面上的一个**矩形区域** $region(v)$ 以及子集 $P(v) = P \cap region(v)$

❖ 树高 $height(T) = \mathcal{O}(\log n)$

❖ 任何**同层**的节点 v 和 u , 都有:

$$region(v) \cap region(u) = \emptyset$$

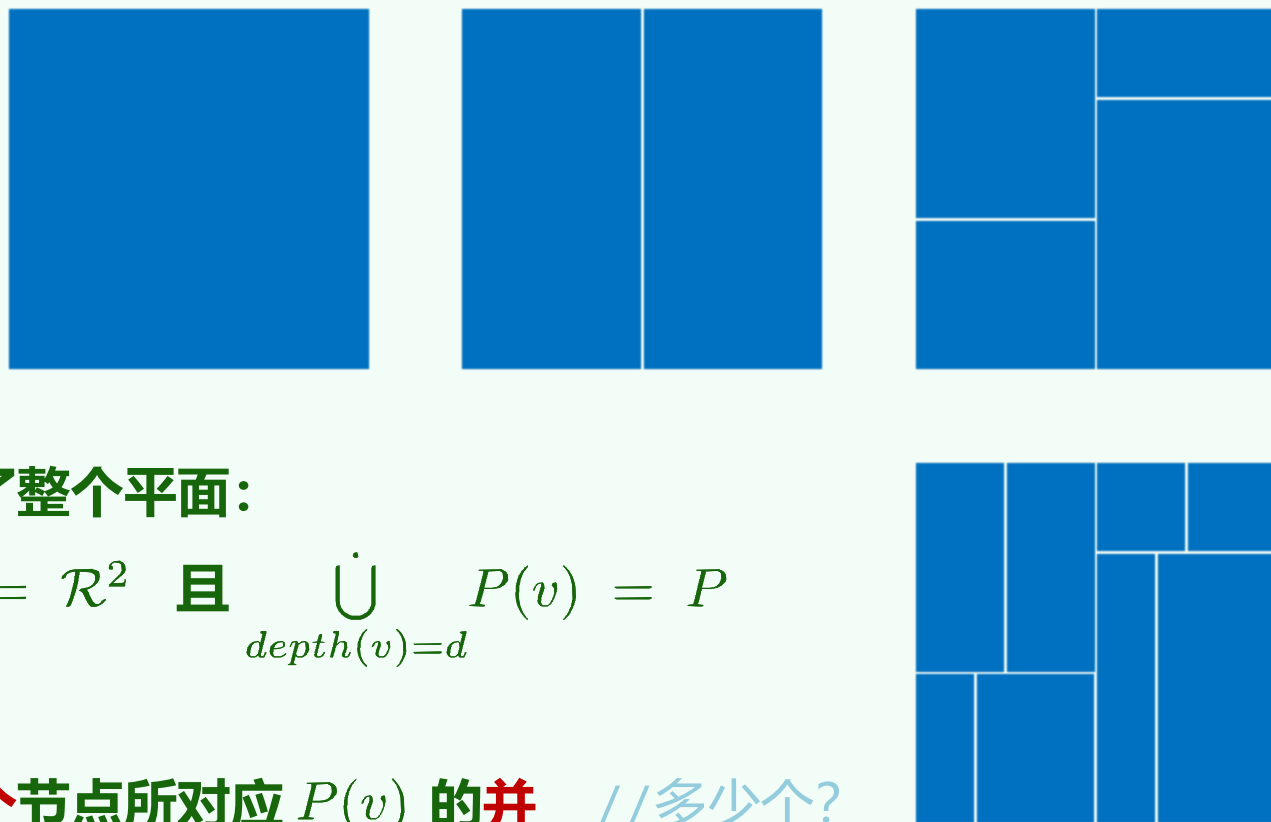
❖ 若**兄弟节点** v 和 u 的父亲为 p , 则有:

$$region(p) = region(v) \dot{\cup} region(u)$$

❖ 同层的**所有**节点所对应的区域, 无缝地**覆盖**了整个平面:

$$\forall 0 \leq d \leq height(T), \quad \bigcup_{depth(v)=d} region(v) = \mathcal{R}^2 \quad \text{且} \quad \bigcup_{depth(v)=d} P(v) = P$$

❖ 任何一次矩形查询的**解**, 都可以表示为**若干个**节点所对应 $P(v)$ 的**并** //多少个?



07-D2

搜索树应用

kd-树：查询

我们竟为这无用的找寻浪费了这么多天！我想找寻的心上人
绝对不会在这里出现

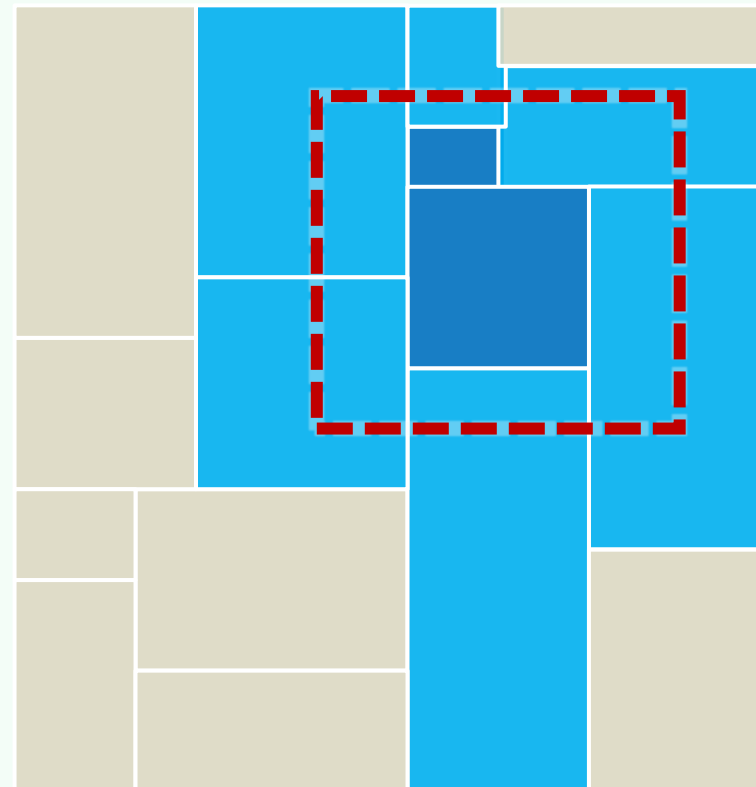
邓俊辉

deng@tsinghua.edu.cn

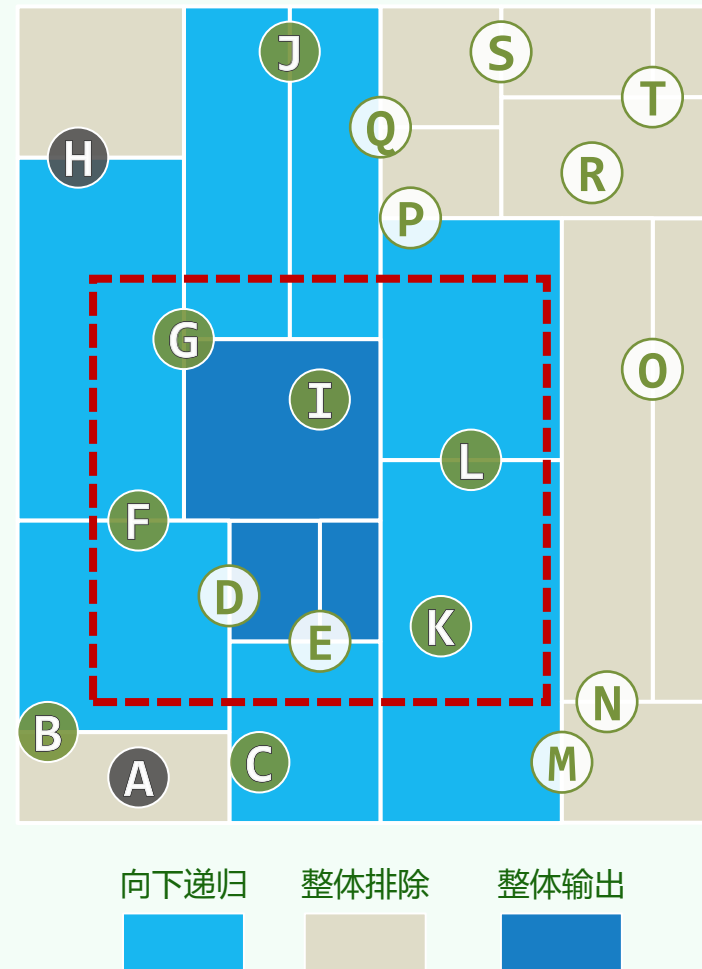
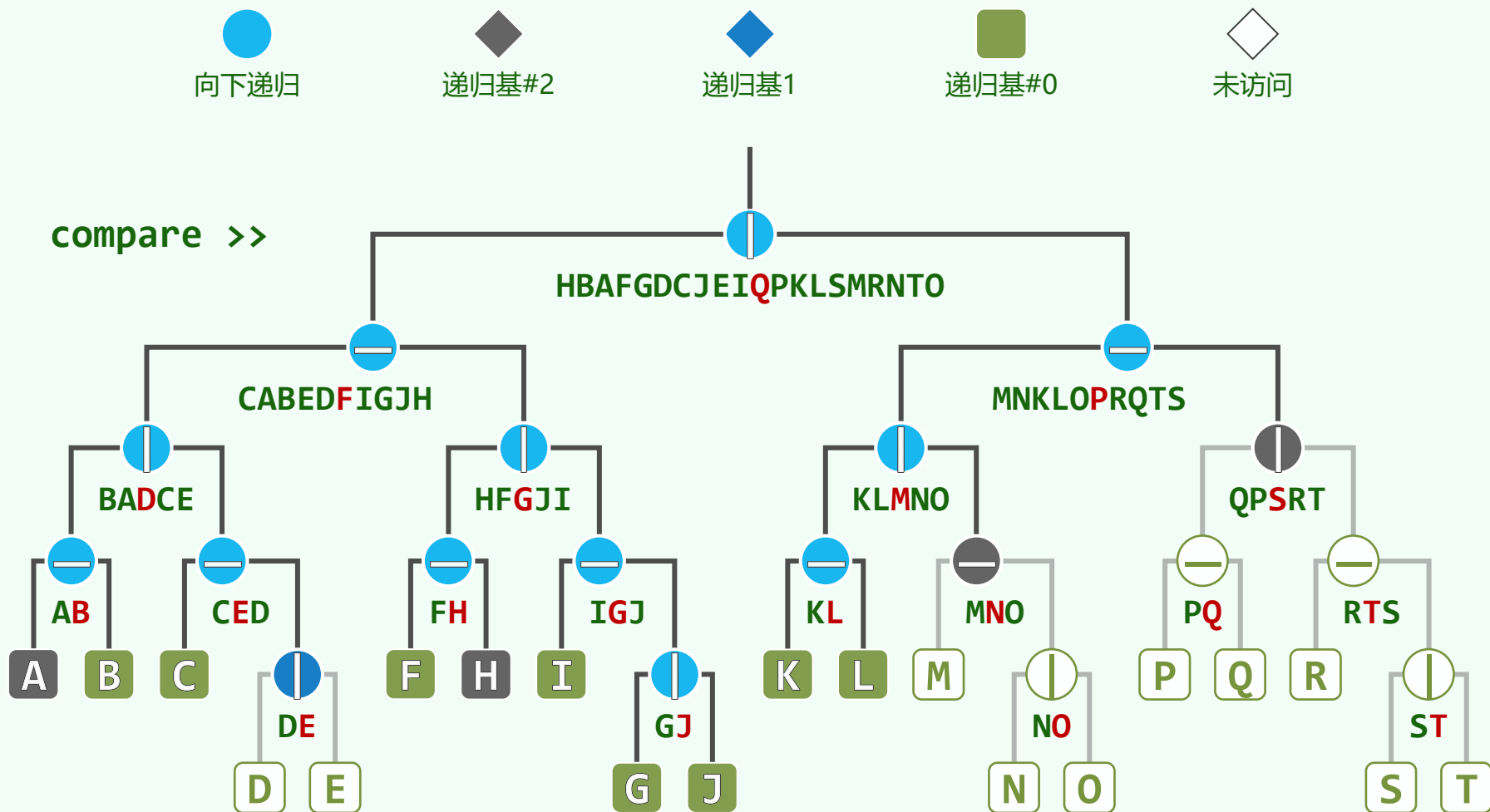
算法

```
kdSearch(v, R) //对节点v做范围R的查询
    if ( isLeaf(v) )
        kdLeaf(v) //递归基#0: 叶子, 单判
    else
        kdCheck(v->lc, R) //向左试探
        kdCheck(v->rc, R) //向右试探

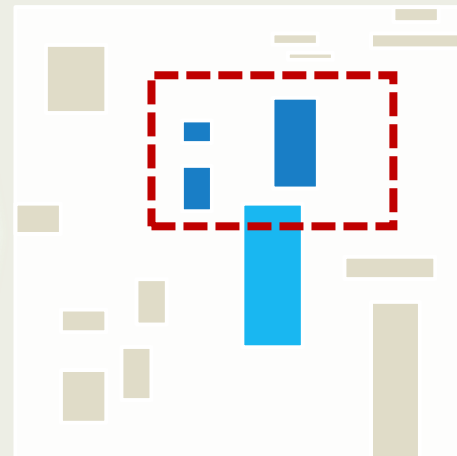
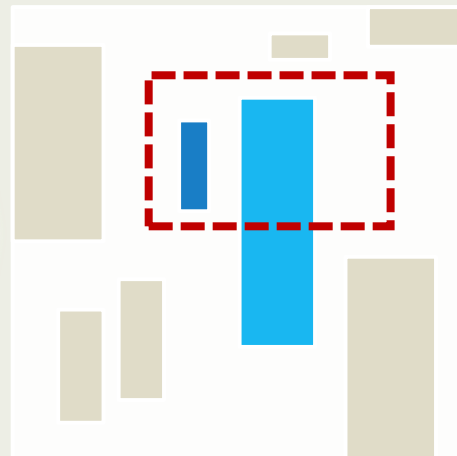
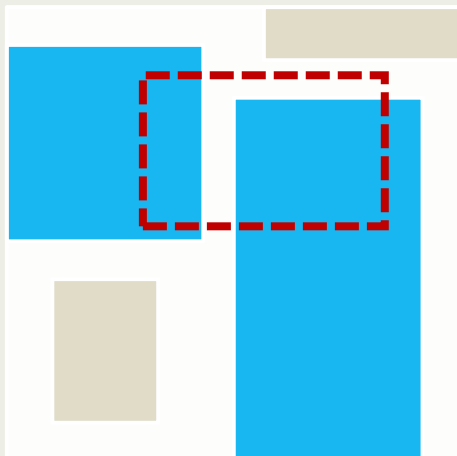
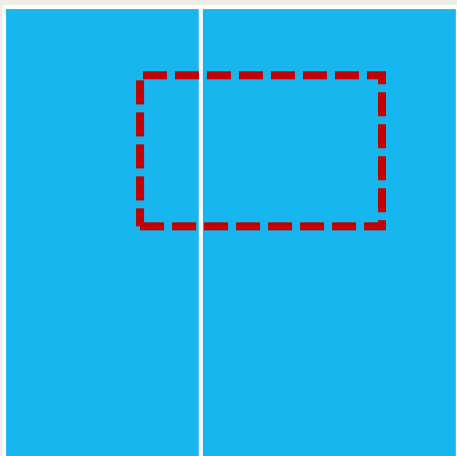
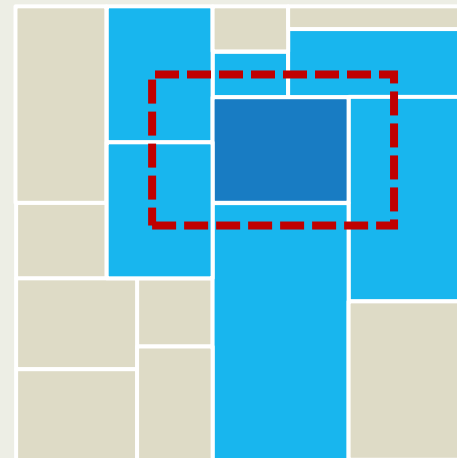
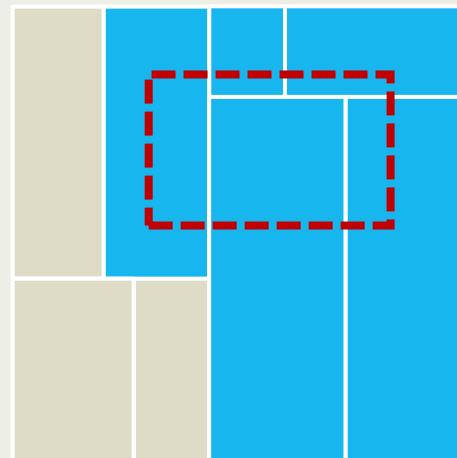
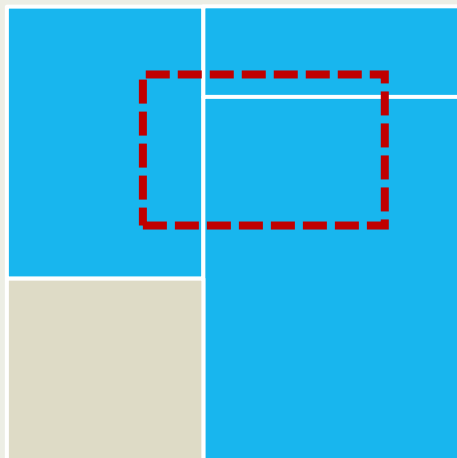
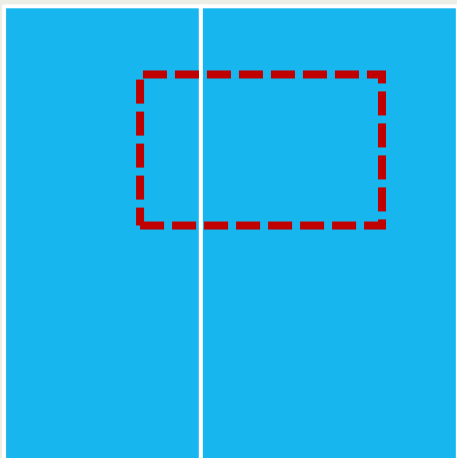
kdCheck(v, R) //试探
    if ( region(v)  $\subseteq$  R )
        kdOutput(v) //递归基#1: 子树整体输出
    else if ( region(v)  $\cap$  R  $\neq \emptyset$  )
        kdSearch(v, R) //向下深入
    //else //递归基#2: 整体排除
```



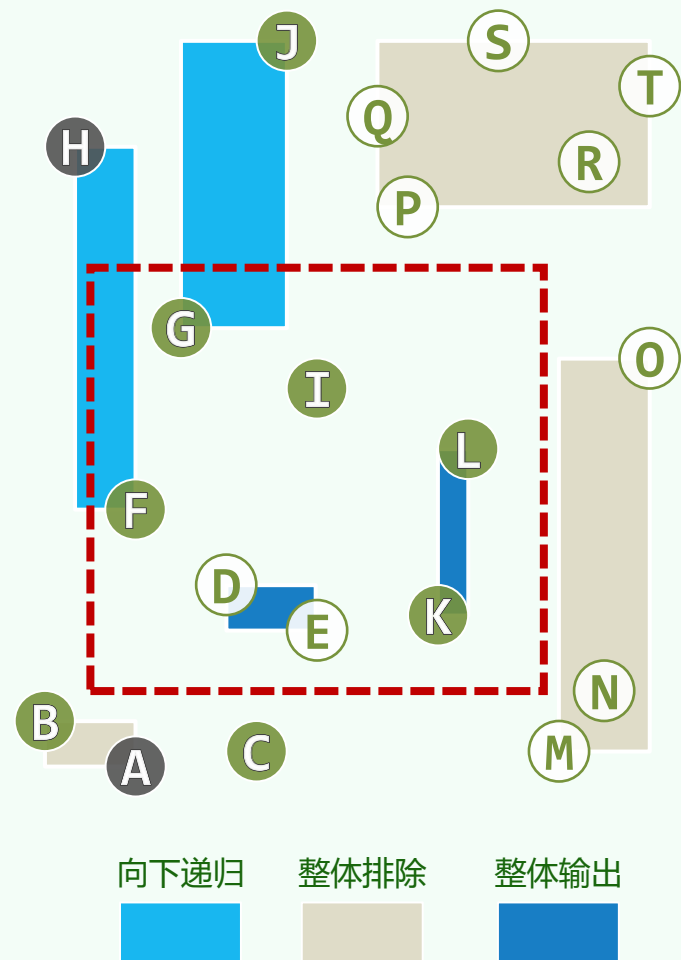
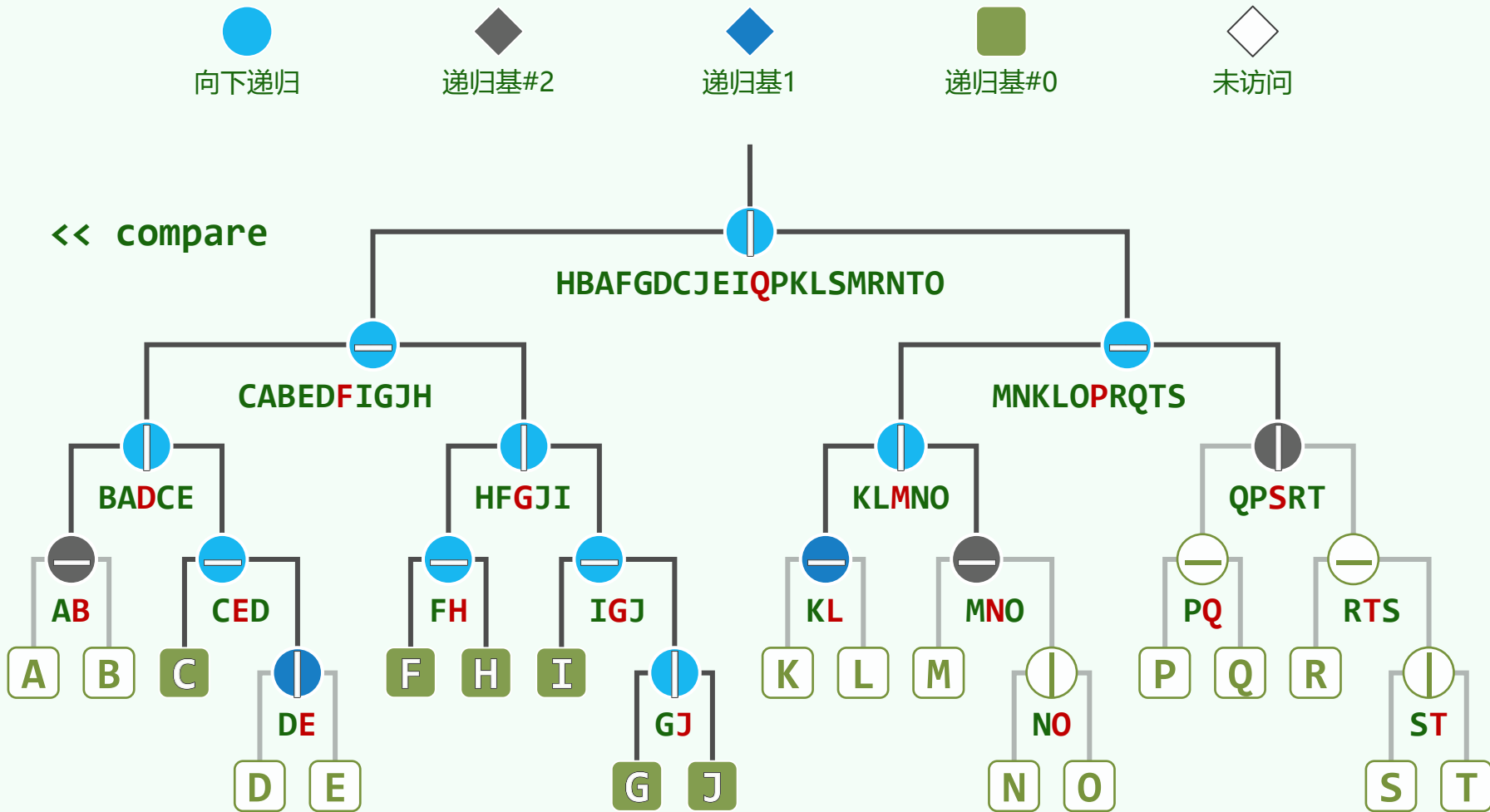
实例：热刀来切 $\log n$ 层巧克力



Bounding Box: 策略



Bounding Box: 实例



搜索树应用

kd-树：复杂度

肉眼看不清细节，但他们都知道那是木星所在的位置，这颗太阳系最大的行星已经坠落到二维平面上了

有人嘲笑这种体系说：为了能发现这个比例中项并组成政府共同体，按照我的办法，只消求出人口数字的平方根就行了

邓俊辉

deng@tsinghua.edu.cn

预处理 + 空间消耗

❖ 时间 $T(n) = 2 * T(n/2) + O(n)$
 $= O(n \log n)$

❖ 树高 = $O(\log n)$

❖ 空间 = 1

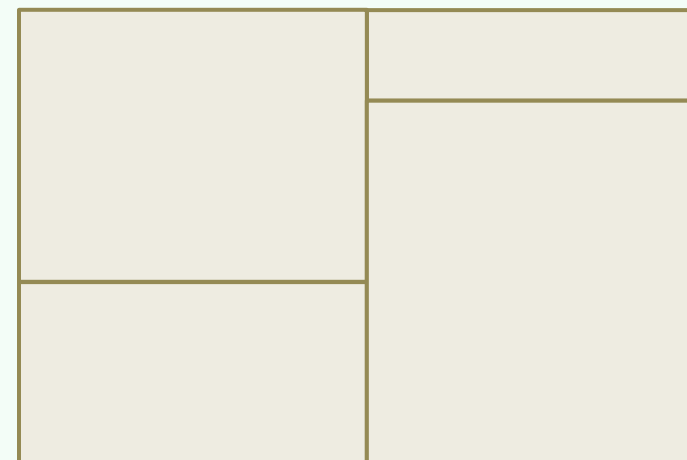
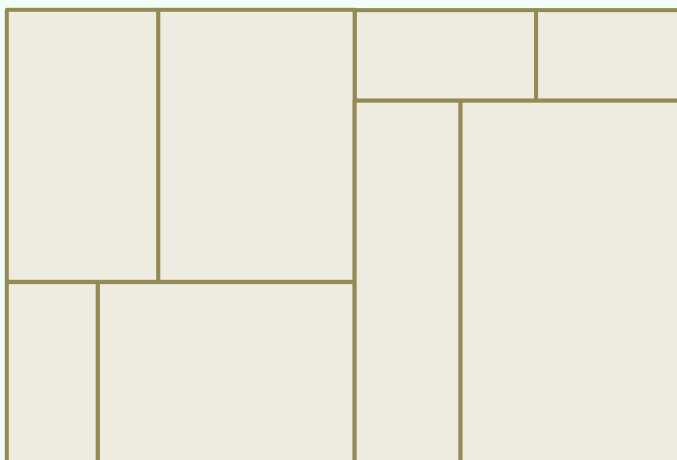
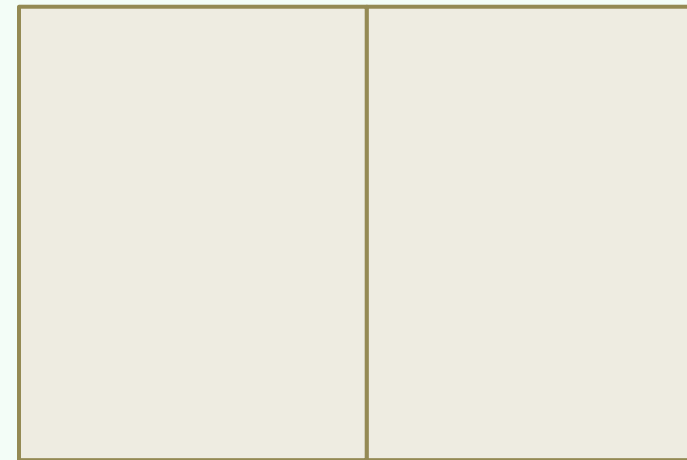
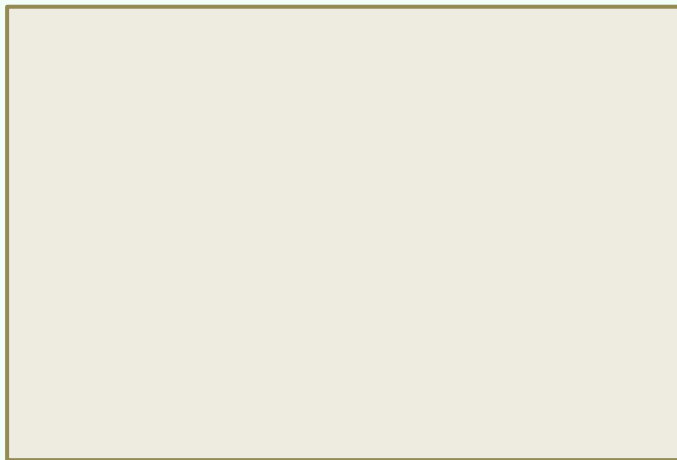
+ 2

+ 4

+ ...

+ $O(2^{\log n})$

= $O(n)$



查询时间 (1/2)

❖ 声明: 查找+报告 = $\mathcal{O}(\sqrt{n} + r)$

❖ 以下均就**最坏情况**而言...

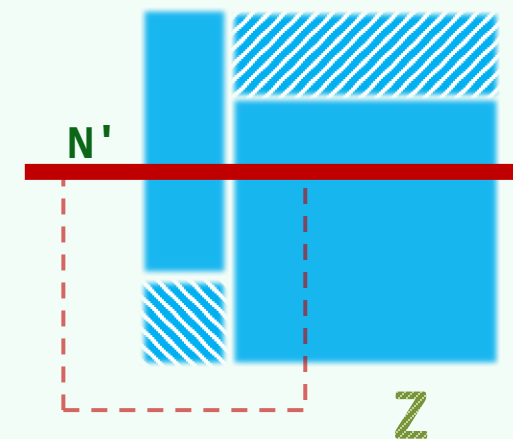
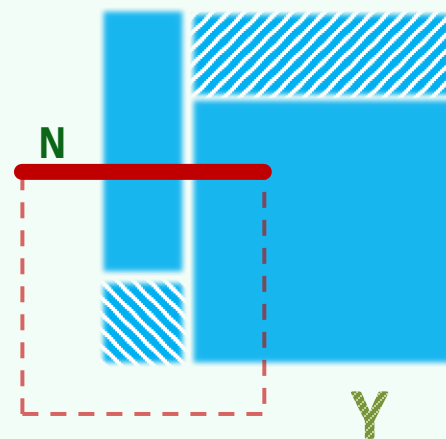
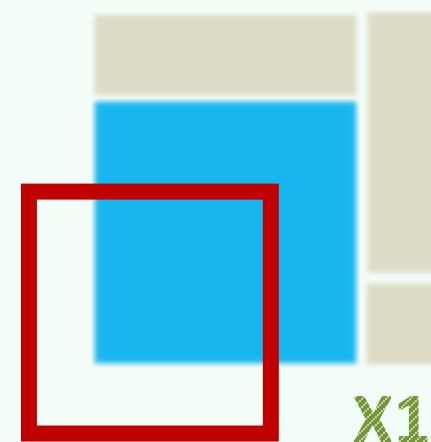
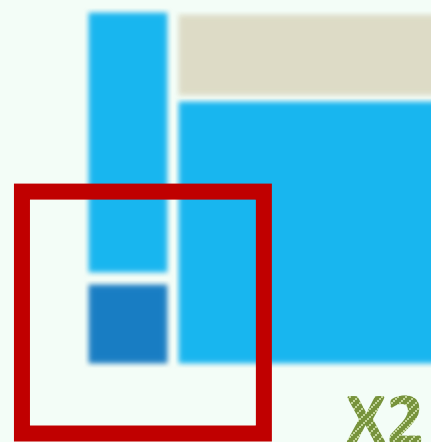
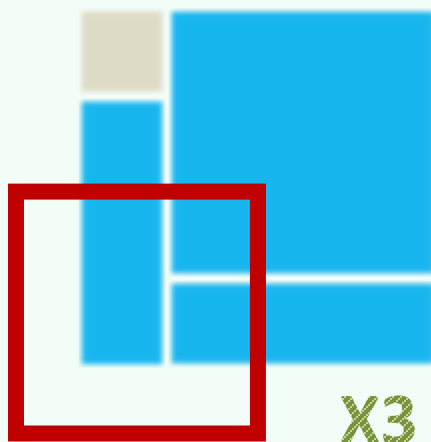
❖ 查找的时间成本, 决定于

- **递归调用的总次数** $Q(n)$, 亦即
- 与**R边界相交**的子区域 (节点) 总数

❖ **任何一段边界**所引发的递归总数, 与 $Q(n)$ 渐近相等

只需考查**其中一段**, WOLG, 北方的那段**N**

❖ 进一步地, 可将**N**放大为一条直线**N'**

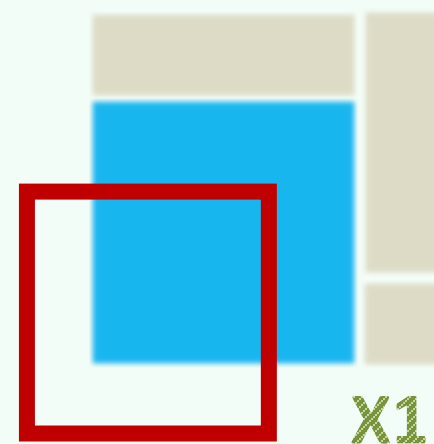
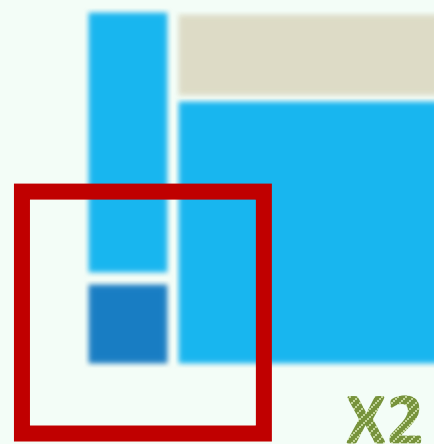
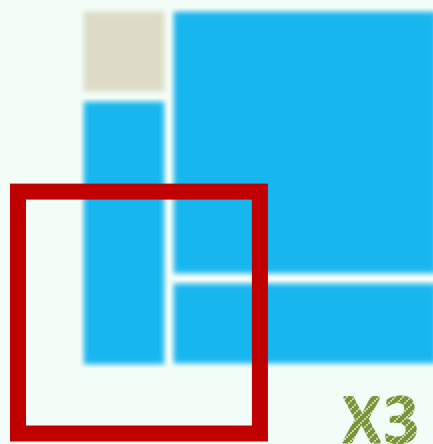


查询时间 (2/2)

❖ 以下，我们来导出 $Q(n)$ 的递推式

❖ 很遗憾，通常由父及子的思路

在这里行不通，需改成由祖及孙

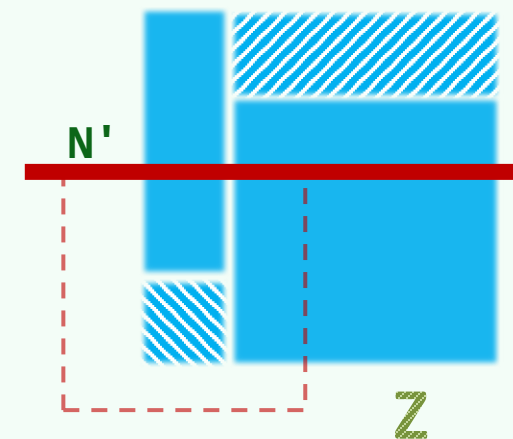
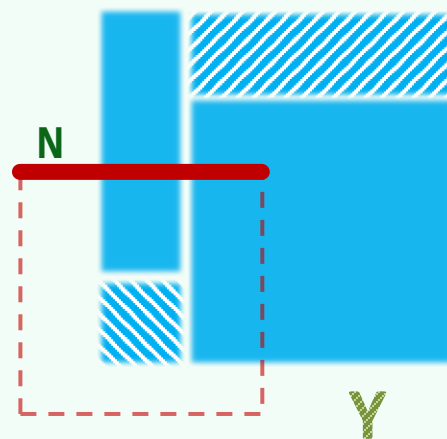


❖ 幸运的是

奇数层上发生递归的总次数，也与 $Q(n)$ 渐近相等

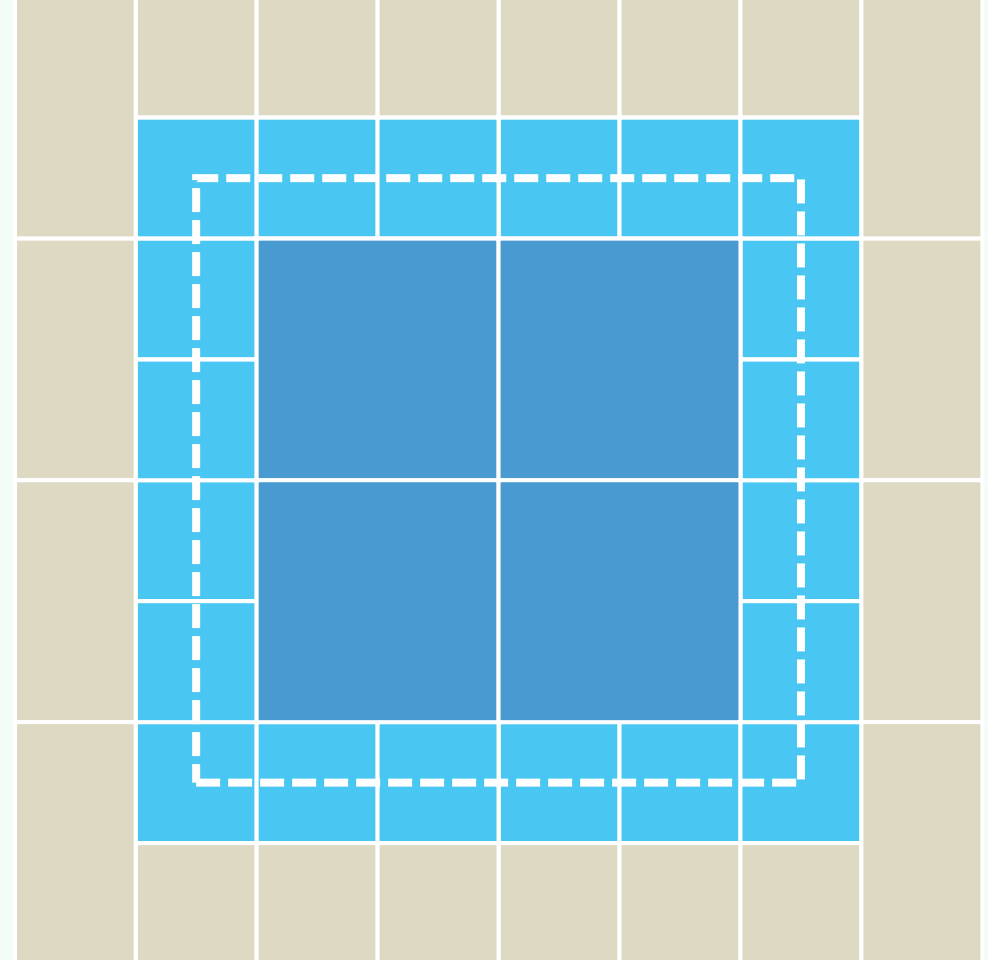
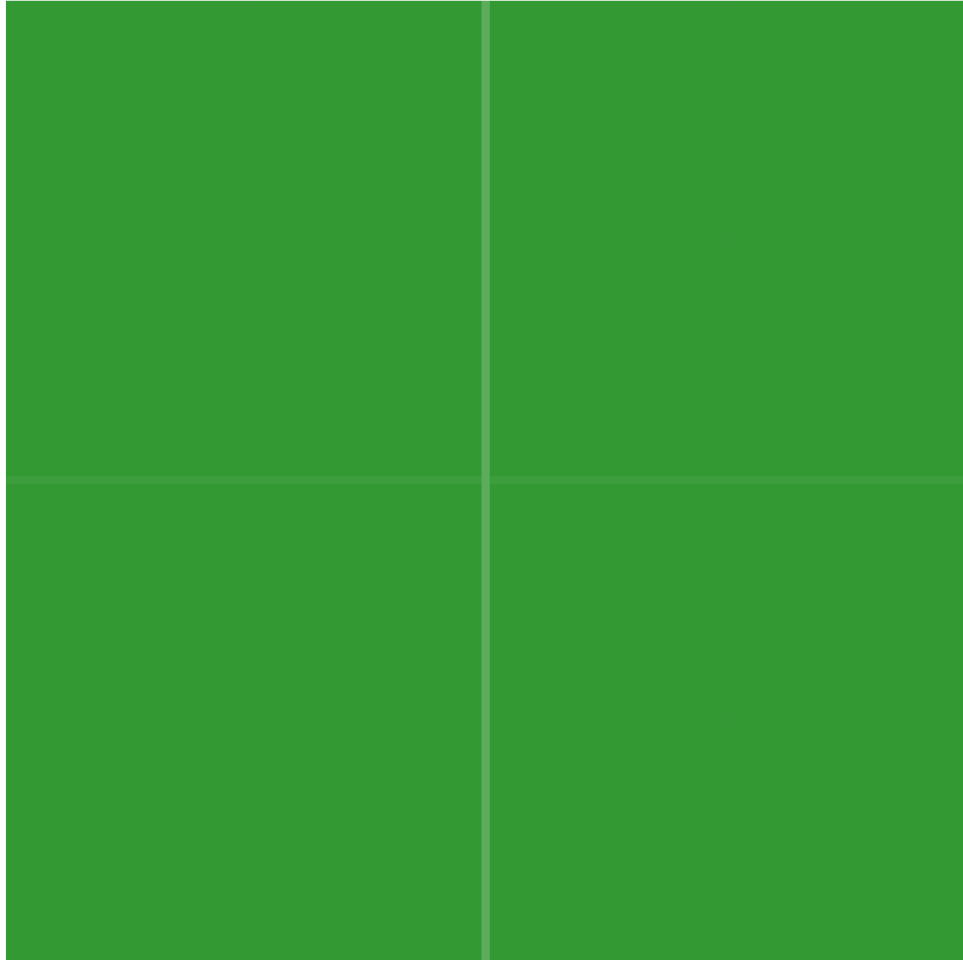
❖ 任一区域若与 N' 有交，则

在其四个孙子中，至多会有两个也与 N' 有交



❖ 亦即: $Q(n) = 2 \cdot Q(n/4) + O(1) = O(\sqrt{n})$

最坏情况



更高维度

❖ 2d-树可否推广，以支持高维空间的范围查询？

在这类场合，其时间、空间效率如何？

❖ 实际上，推广并不难：

在k维空间中，只需循环地依次沿第1、第2、第3、...、第k维切分

❖ 在d维欧氏空间 $\mathcal{E}^d$ 中，对于任意给定的 n 个点，我们都可以

- 在 $\mathcal{O}(n \log n)$ 时间内构造出
- 一棵占用 $\mathcal{O}(n)$ 空间的kd-树，并借助它
- 在 $\mathcal{O}(r + n^{1-1/d})$ 时间内完成每次正交范围查询